

บทที่ 4

การทดลองและผลการทดลอง

4.1. โครงร่างของระบบ

4.1.1. ฮาร์ดแวร์และซอฟต์แวร์ที่ใช้ในการทดลอง

เครื่องคอมพิวเตอร์ที่ใช้ในการทดลอง

ASP .Net เว็บแอปพลิเคชันเซิร์ฟเวอร์

หน่วยประมวลผลกลาง	: AMD Sempron 2000+
หน่วยความจำ	: 512 MB
ฮาร์ดดิสก์	: 80 GB
ระบบปฏิบัติการ	: Windows OS
ระบบฐานข้อมูล	: PostgreSQL version 8.0
เซิร์ฟเวอร์	: IIS version 5.1
툴ที่ใช้พัฒนา	: Microsoft Visual Studio .Net 2003
Plug-in	: WSE 1.0

Java Web Service Server

หน่วยประมวลผลกลาง	: Pentium4 1.7 GHz
หน่วยความจำ	: 512 MB
ฮาร์ดดิสก์	: 120 GB
ระบบปฏิบัติการ	: Windows OS
เซิร์ฟเวอร์	: Websphere Application Server 5.1
툴ที่ใช้พัฒนา	: Websphere Application Developer 5.1.1

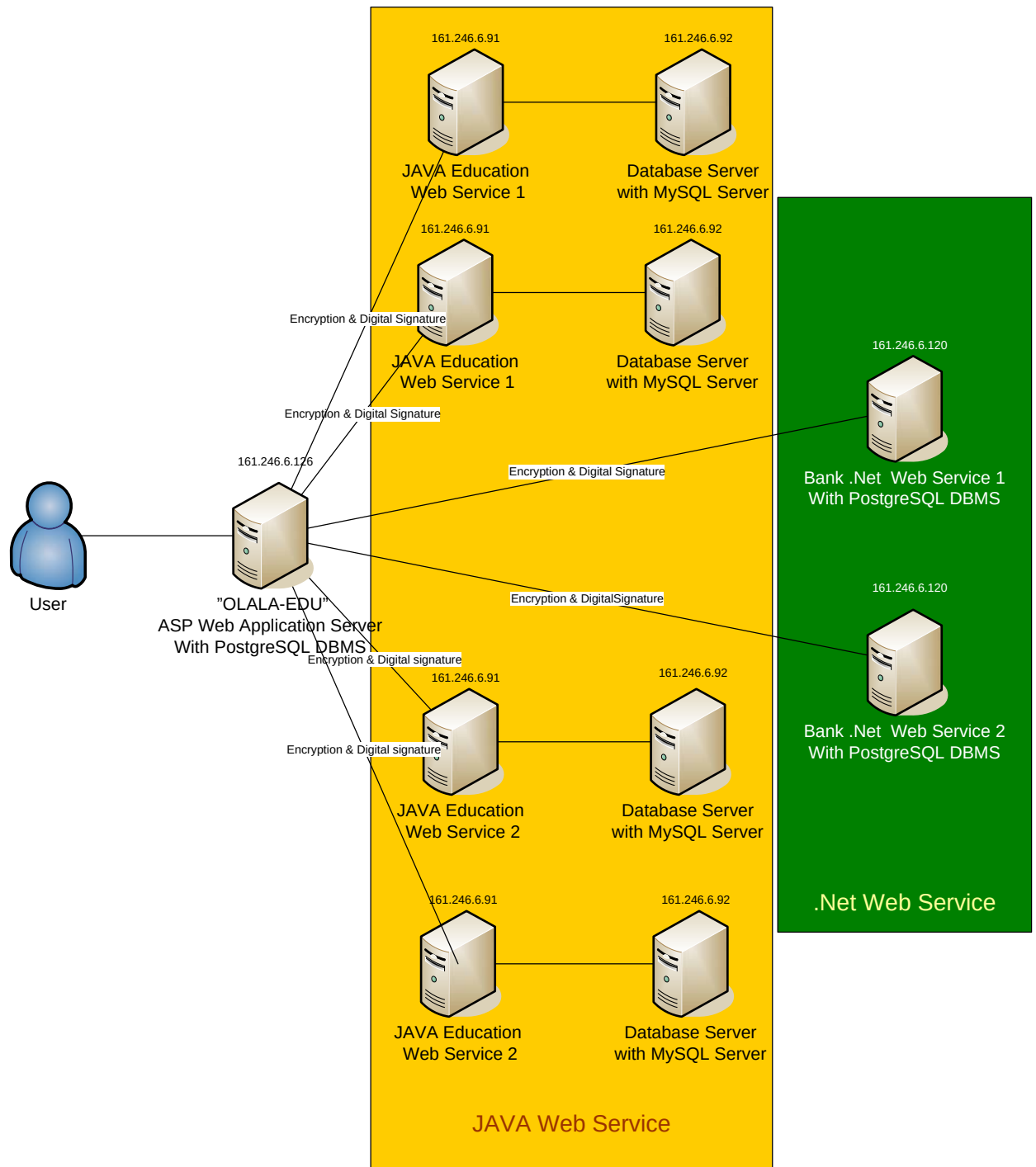
Database Server

หน่วยประมวลผลกลาง	: Pentium M 1.4
หน่วยความจำ	: 512 MB
ฮาร์ดดิสก์	: 40 GB
ระบบปฏิบัติการ	: Windows OS
ระบบฐานข้อมูล	: MySQL version 3.2 build 6.14
툴ที่ใช้พัฒนา	: Eclipse 3.1

.Net Web Service Server

หน่วยประมวลผลกลาง	: Pentium 4 3.0 GHz
หน่วยความจำ	: 1 GB
ฮาร์ดดิสก์	: 120 GB
ระบบปฏิบัติการ	: Windows OS
ระบบฐานข้อมูล	: PostgreSQL version 8.0
เซิร์ฟเวอร์	: IIS version 5.1
ทูลที่ใช้พัฒนา	: Microsoft Visual Studio .Net 2003
Plug-in	: WSE 2.0

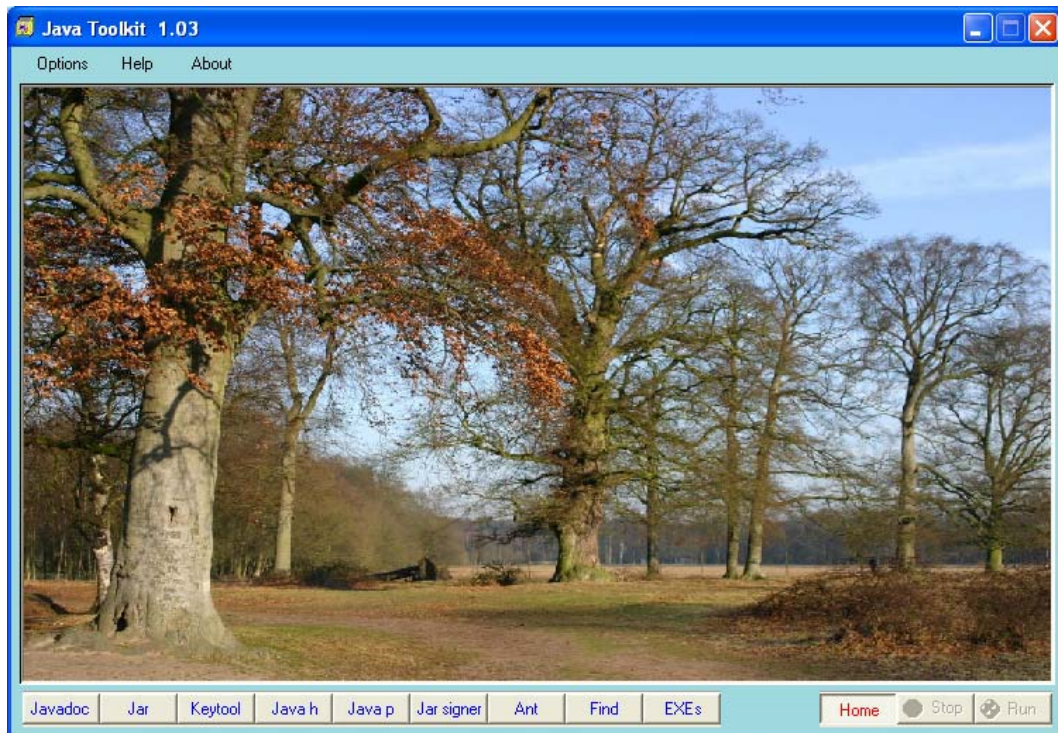
4.1.2. แบบจำลองระบบที่มีการนำ WS-Security ไปใช้



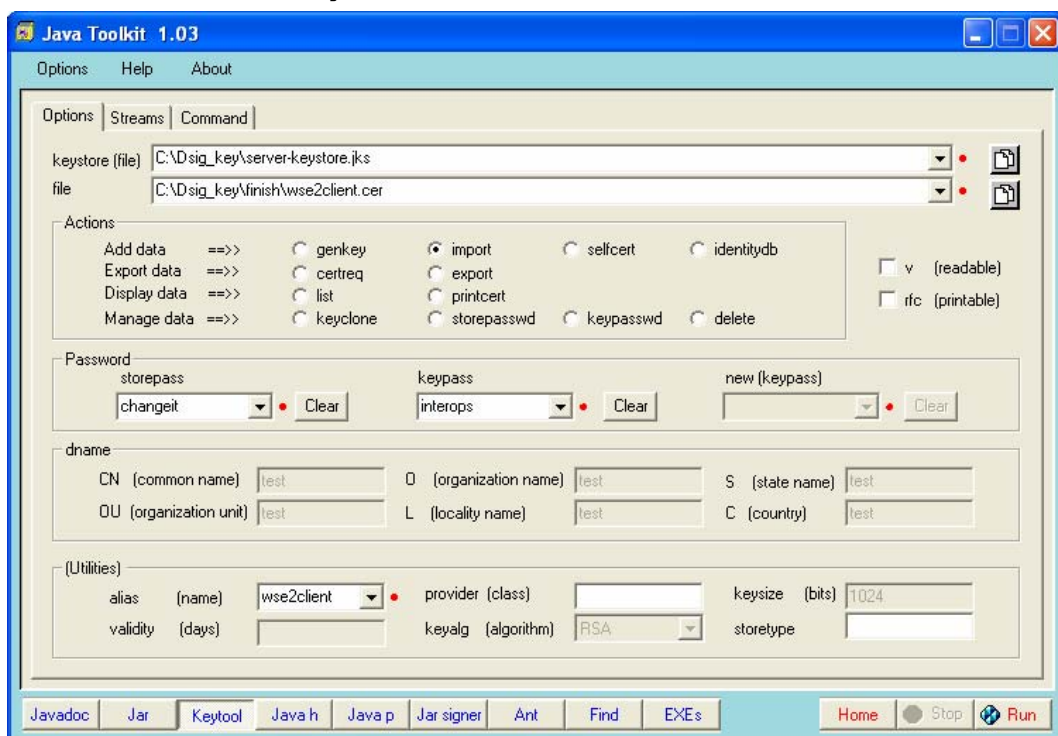
รูปที่ 4.1 แสดงแบบจำลองระบบที่มีการนำ WS-Security ไปใช้

4.2. ผลการทดลอง

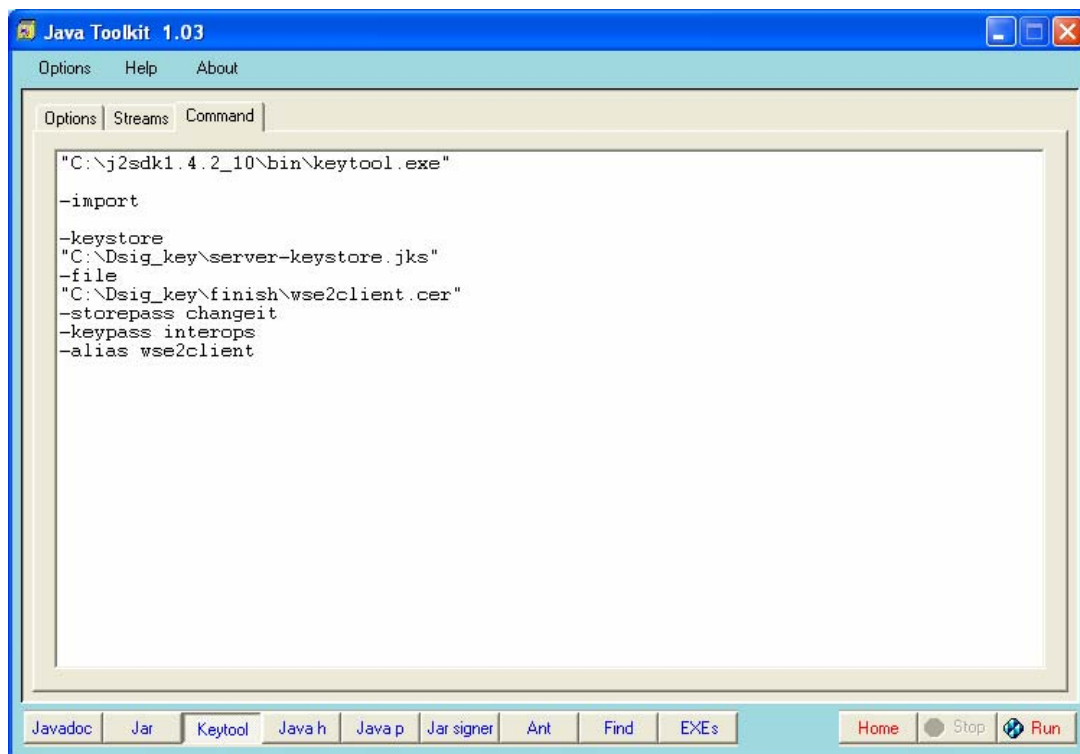
4.2.1. การสร้างคีย์สำหรับใช้ในเว็บเซอร์วิสด้วย Java Toolkit 1.03



รูปที่ 4.2 แสดง โปรแกรม Java Toolkit



รูปที่ 4.3 แสดงการสร้างคีย์ด้วยโปรแกรม Java Toolkit



รูปที่ 4.4 แสดงผลการสร้างคีย์ด้วย โปรแกรม Java Toolkit เป็น command

คีย์ที่ใช้ในการจัดการเรื่องความปลอดภัยของเว็บเซอร์วิสมีหลายรูปแบบซึ่งคีย์ที่ใช้ร่วมกันได้มีเพียงคีย์รูปแบบ PKCS12 เท่านั้นที่ใช้ได้ทั้งคู่ ซึ่งสามารถอธิบายรูปแบบการสร้าง อย่างง่ายๆ ได้ ดังนี้

ตารางที่ 4.1 แสดงรูปแบบการสร้าง คีย์แบบต่างๆ

ชนิดของคีย์	รูปแบบการสร้าง	การนำไปใช้
JKS	1. สร้างเองจาก keytool –genkey 2. import จากคีย์ชนิดอื่น โดยใช้ keytool –import	Java Platform เท่านั้น
JCEKS	1. สร้างเองจาก keytool –genkey 2. import จากคีย์ชนิดอื่น โดยใช้ keytool –import	Java Platform เท่านั้น
PKCS12	สร้างจาก Tool เฉพาะซึ่งมีความน่าเชื่อถือ เช่น OpenSSL	Java & .Net สามารถใช้ได้
PFX	1. สร้างเองจาก makecert tool 2. ทำการconvert key ของ Java โดยใช้ OpenSSL	.Net เท่านั้น
CER (public key)	1. สร้างได้จาก makecert tool 2. สามารถขอคีย์จาก CA ได้	Java & .Net สามารถใช้ได้

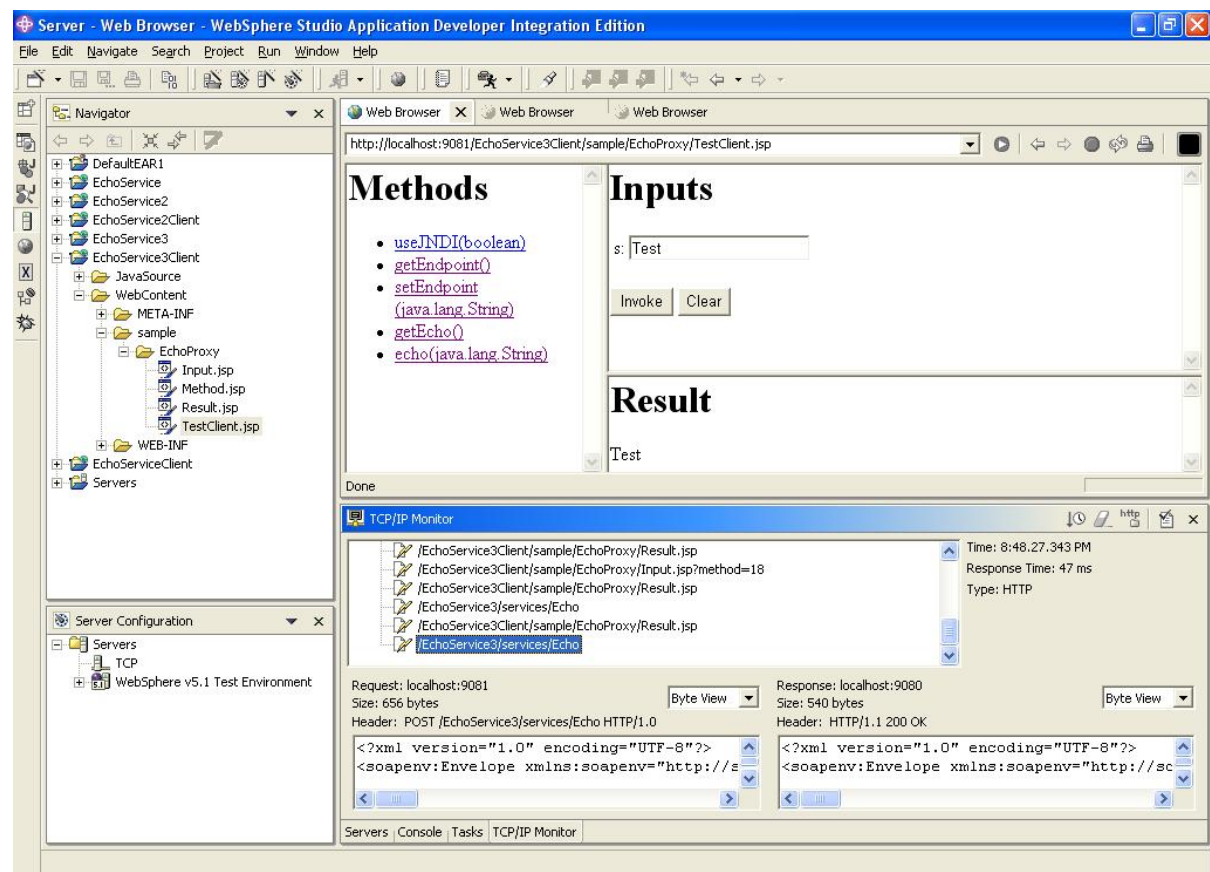
1. คีย์ที่จะนำไปใช้ใน Java web service ได้นั้นต้องถูกบรรจุใน “Keystore” ชนิด JKS, JCEKS, PKCS12 เท่านั้นโดยสามารถสร้างขึ้นเองโดยใช้ keytool -genkey หรือ นำคีย์ที่มีอยู่แล้ว import เข้าไปใน keystore ก็ได้

2. ลิงก์ที่นำไปใช้ใน .Net Webservice ต้องเป็นคีย์ที่ถูกบรรจุใน “Keystore” ชนิด PKCS12, PFX หรือ public key certificate โดยสามารถสร้างจาก makecert หรือ อาจนำคีย์ของจาวา มาใช้โดยใช้ OpenSSL ในการแปลงคีย์

4.2.2. การเซตค่าเพื่อให้สอดคล้องกับหลักการ WS-Security ระหว่าง Java Web service และ Java Webservice โดยใช้ Websphere Application Developer 5.1.1

จากทฤษฎีเกี่ยวกับเว็บเซอร์วิสที่ผ่านมาได้ทำการทดลองใช้งานในส่วนของ WS-Security ซึ่ง แสดงให้เห็นใน SOAP ได้ดังตัวอย่าง

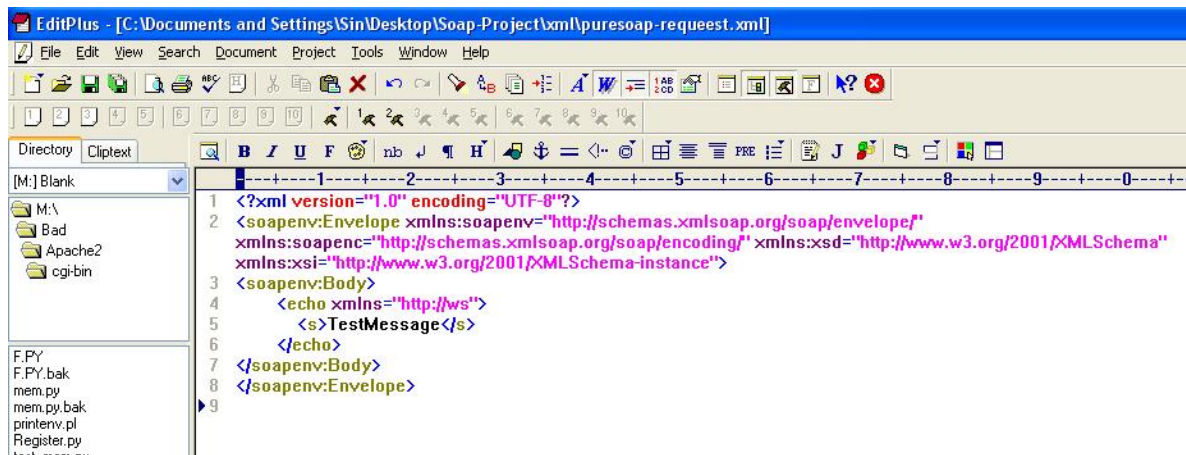
1.2.2.1. การเพิ่มความปลอดภัยในการพิสูจน์ตัวตน (Authentication) โดยใช้ Username Token

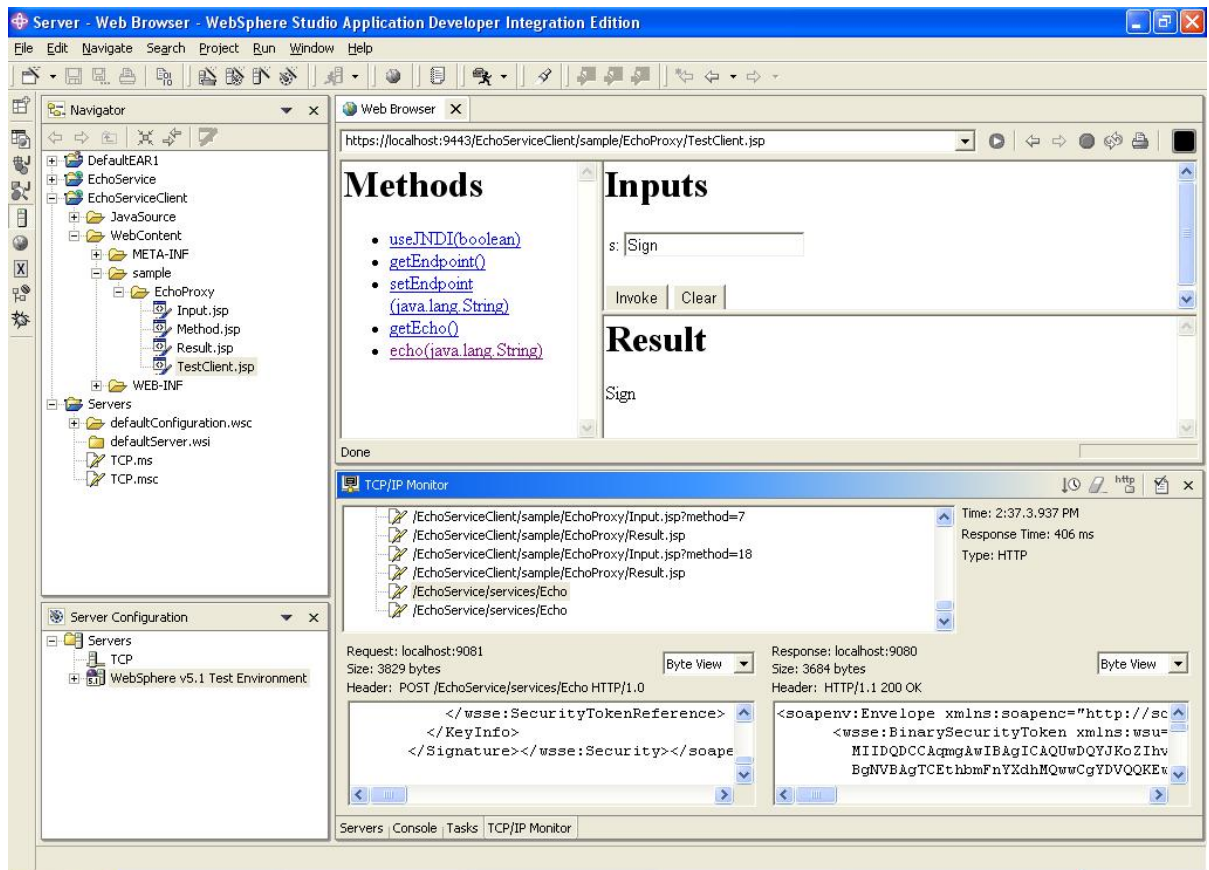


รูปที่ 4.5 แสดง ตัวอย่างโปรแกรม Echo Service ที่มีการทำ Username Token

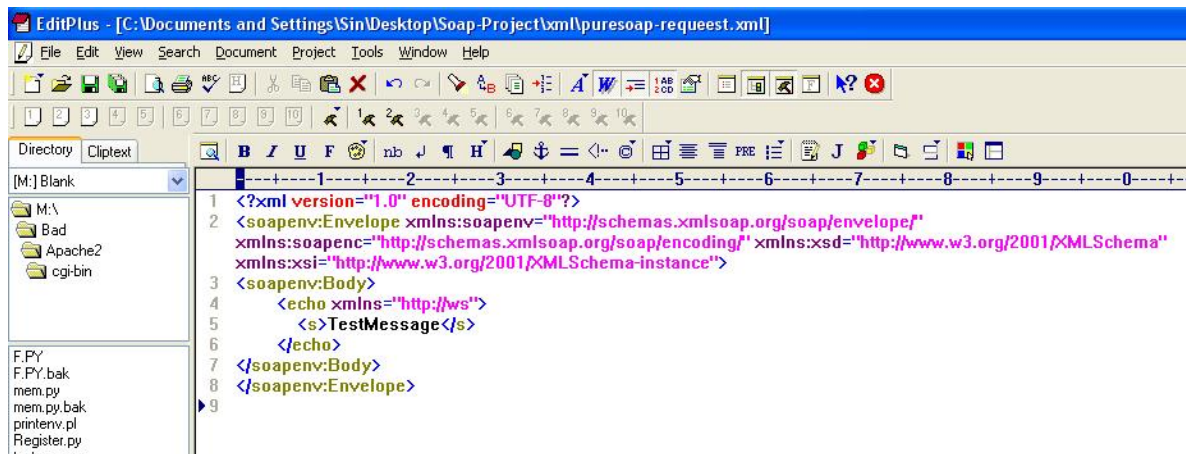
ใน SOAP ที่มีการทำ Username Token แล้วจะมี Tag **<wsse: usernameToken>** แสดงถึงการใช้ Username Token ใน SOAP นั้น เพื่อหลีกเลี่ยงการโจมตี จึงมีการสร้าง element **<wsse:Nonce>** และ **<wsu:Created>** ไว้ด้านใน **<wsse: usernameToken>** element และใช้ในการ

รับรอง message ในการร้องขอผู้รับหรือการตอบกลับผู้รับ จะทำการตรวจสอบ ความบริสุทธิ์ ว่ามีความแตกต่างจาก message ที่ถูกสร้างไว้ก่อนหน้านี้หรือไม่





รูปที่ 4.8 แสดงตัวอย่างโปรแกรม Echo ที่มีการทำ Digital Signature แล้ว



รูปที่ 4.9 แสดง SOAP Message ที่ไม่มีการ ทำ Digital signature


```

1 <soapenv:Envelope xmlns:soapenc="http://schemas.xmlsoap.org/soap/encoding/"
  xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/"
  xmlns:xsd="http://www.w3.org/2001/XMLSchema"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" >
2 <soapenv:Header>
3 <wsse:Security xmlns:wsse="http://schemas.xmlsoap.org/ws/2003/06/secext"
  soapenv:actor="myActorURI" soapenv:mustUnderstand="1" >
4 <wsse:BinarySecurityToken xmlns:wsu="http://schemas.xmlsoap.org/ws/2003/06/utility"
  EncodingType="wsse:Base64Binary" ValueType="wsse:X509v3"
  wsu:Id="wssecurity_binary_security_token_id_408039426707555435" >
5 MIIDQTCCAqggAwIBAgICAQAwDQYJKoZIhvcNAQEFBQAwTjELMAkGA1UEBhMCSEAxEtAP
6 BgNVBAGTCeIthbmFnYXdhMQwwCgYDVQQKEwNjQk0xDDAKBgNVBAStA1RSTDEQMA4GA1UE
7 AxMHSW50IENBMjAeFw0wMTEwMDEwOTU0MDZaFw0xMTEwMDEwOTU0MDZaMFQxCzAJBgNV
8 BAYTAkpQMREwDwYDVQQIEWhlYW5hZ2F3YTEEMAAoGA1UEChMDSUJNMQwwCgYDVQQLEwNU
9 UkwxvFJAUBgNVBAMTDVNPQVBSZXI1ZXN0ZXIwZ8wDQYJKoZIhvcNAQEBBQADgY0AM1GJ
10 </wsse:BinarySecurityToken>
11 <Signature xmlns="http://www.w3.org/2000/09/xmldsig#" >
12 <SignedInfo>
13 <CanonicalizationMethod
14 Algorithm="http://www.w3.org/2001/10/xml-exc-c14n#" ></CanonicalizationMethod>
15 <SignatureMethod
16 Algorithm="http://www.w3.org/2000/09/xmldsig#rsa-sha1" ></SignatureMethod>
17 <Reference URI="#wssecurity_body_id_4508207003845519626" >
18 <Transforms>
19 <Transform Algorithm="http://www.w3.org/2001/10/xml-exc-c14n#" >
20 <ec:InclusiveNamespaces xmlns:ec="http://www.w3.org/2001/10/xml-exc-c14n#"
21 PrefixList="xsi soapenc xsd #default wsu soapenv" ></ec:InclusiveNamespaces>
22 </Transform>
23 </Transforms>
24 <DigestMethod Algorithm="http://www.w3.org/2000/09/xmldsig#sha1" ></DigestMethod>
25 <DigestValue>zn0oiAnV9R9kA61Si6BZaQondfO= </DigestValue>
26 </Reference>
27 </SignedInfo>
28 <SignatureValue>
29 g1ErZ1zUQMjX14OLktdrLLpfAwI7XrJes9GoXaLnT7zenPRjrHMgmNnLfN5
30 a/VV0SqTYFhX6vJMukPFui83jTVvdzPeT8/BoOFwkr/Wsr8VdKS5fUxcPd9
31 Ve0RcC+zkGV2rfcS33+EbVXaxKVhSS6147ipGU6ENaueMXDzWT8=
32 </SignatureValue>
33 <KeyInfo>
34 <wsse:SecurityTokenReference>
35 <wsse:Reference
36 URI="#wssecurity_binary_security_token_id_408039426707555435" ></wsse:Reference>
37 </wsse:SecurityTokenReference>
38 </KeyInfo>
39 </Signature>
40 </wsse:Security>
41 </soapenv:Header>
42 <soapenv:Body xmlns:wsu="http://schemas.xmlsoap.org/ws/2003/06/utility"
  wsu:Id="wssecurity_body_id_4508207003845519626" ><echo xmlns="http://ws" ><s>Test
  Message</s></echo>
43 </soapenv:Body>
44 </soapenv:Envelope>

```

รูปที่ 4.10 แสดง SOAP Message ที่มีการ ใช้ Digital signature

จากภาพแสดงให้เห็นส่วนประกอบที่เพิ่มขึ้นมาหลังจากมีการ ทำ Digital signature แล้ว คือ ในส่วนของ element ต่างๆ

< Signature >

เป็นส่วนที่เก็บข้อมูลทั้งหมดโดยกล่าวถึง Information ที่ใช้ในการ sign เช่น signature, key ที่ใช้ sign, ตำแหน่งที่ใช้ในการเก็บ information

< SignedInfo >, < SignatureMethod > , < CanonicalizationMethod >

เป็นส่วนที่ระบุชนิดของ Signature เช่น Kerberos หรือ RSA ซึ่งใช้ในการสร้าง signature element เหล่านี้ เป็นส่วนที่แสดงให้เห็นว่ามีการแบ่งการทำงานเป็นส่วนๆ และ สามารถแสดงให้เห็นถึงการป้องกันการเปลี่ยนแปลงแก้ไขข้อมูล จากบุคคลที่ 3

< SignatureValue >

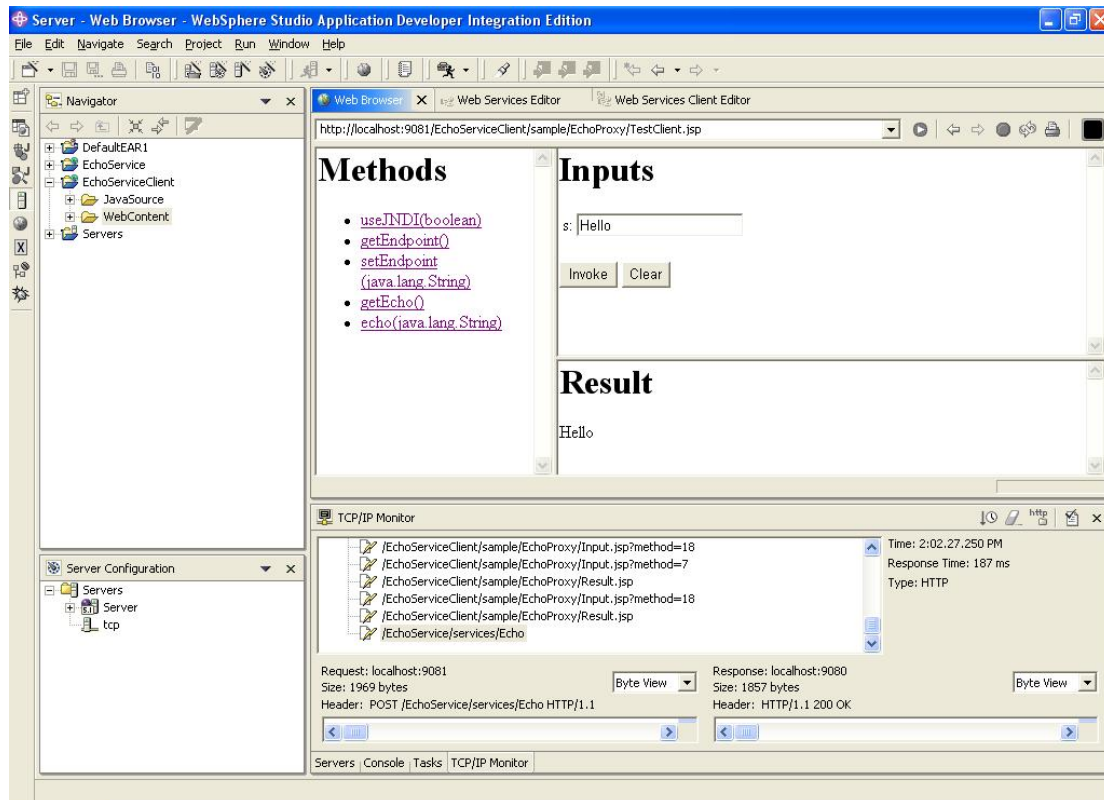
เป็นส่วนที่เก็บข้อมูลรายละเอียดของ Application

< KeyInfo >

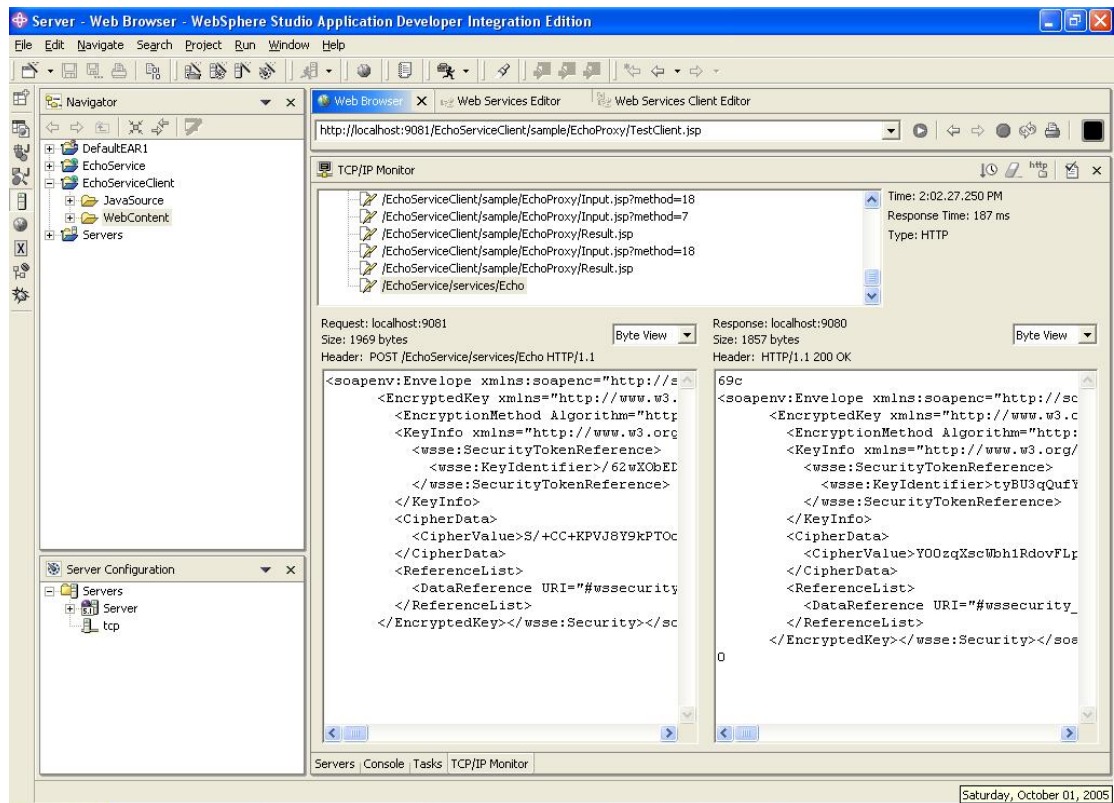
เป็นส่วนที่ระบุชื่อ หรือ Key ที่ใช้ sign

4.2.2.3. การเพิ่มความปลอดภัยของข้อมูลที่เป็นความลับ (Confidentiality) โดยการใช้ XML

Encryption 3รูปแบบ



รูปที่ 4.11 แสดง การ ส่ง Echo Message ที่มีการ Encryption SOAP เข้าไปด้วย



รูปที่ 4.12 แสดง การ ส่ง Echo Message ที่มีการ Encryption SOAP เข้าไปด้วย

Order.xml

10/3/2005 1:44 PM

```

1  <?xml version="1.0"?>
2  <!-- This Listing provides the sample XML File that will be encrypted. -->
3
4  <purchaseOrder>
5    <Order>
6      <BookName>
7        SoccerForDummies
8      </BookName>
9      <Id>123-958-74598</Id>
10     <Quantity>500</Quantity>
11   </Order>
12
13   <Payment>
14     <CardNo>4502-3456-3278-2011</CardNo>
15     <CardType>VISA</CardType>
16     <ValidDate>12-10-2004</ValidDate>
17   </Payment>
18
19 </purchaseOrder>

```

รูปที่ 4.13 แสดง ตัวอย่าง XML ที่มีหลาย element

4.2.2.3.1. การเข้ารหัสทั้งเอกสาร

```
1 <?xml version="1.0" encoding="UTF-8"?>
2
3 <EncryptedData xmlns="http://www.w3.org/2001/04/xmlenc#" Id="Test"
  Type="http://www.iso.edu/in-notes/iana/assignments/media-types/text/xml">
4   <EncryptionMethod Algorithm="http://www.w3.org/2001/04/xmlenc#tripledes-cbc" />
5   <ds:KeyInfo xmlns:ds="http://www.w3.org/2000/09/xmldsig#">
6     <KeyName>theKey</KeyName>
7   </ds:KeyInfo>
8   <CipherData>
9     <CipherValue>v0oy2cty7WLj5RDNbV2o3UG/BtTt08IWWXf0Y/EHu/qivSa/p7wXP03mSXvtaGY87+
10    hVjCrrH4mp
11    cKr+0xrTJumhVKHyptnHTpJksHxehqM9tCZyQDR0rZeLo76KrU2/FDS2ZDS3SLYDda3PUf5v0paU
12    k2TdoCJIttXPwvbvULpzvP8/5BYVT+t8bqm24YEC9Q8shlmAc/OdXV03zfWlMKoqAZj1je03aImR
13    BobTnqd1o+ilImFUEY6Dmg0pdwDuu+CsN9J3wE3C4co6ISh6zEqLFWmJA2wnytGTi6/t24fzRHeK
14    7f+SOIC+7yLjnnHPBfpvp2ciC/IEg5jHd2qg4vhSKN0k6qfJUJpc5M4lk4dE79lv7LpF5m22tf8
15    z1pDXmwgki1YVi3ksIPXmHALil/y4ggx4u2pCAFR17Bk4bXlRL4lNpoJx/ykL8JfidBrGvKiuYB
16    nbdNcn/8jKzRUyyTfcgHQXFCeUe7JrueiQJjiTIPwl3Ye8u4fipDfegCnbV/rb5/a7vFe2Z/RfRq
17    xUPBJQ9K0uP4+DMTP3qAfKL8F0qgQ5UnvnbLFYNxnx4D</CipherValue>
18   </CipherData>
19 </EncryptedData>
20
```

รูปที่ 4.14 แสดง SOAP Message มีการเข้ารหัสทั้งเอกสารแบบ triple-DES

4.2.2.3.2. การเข้ารหัสเฉพาะ Element

```
4.xml 10/3/2005 1:53 PM
1 <?xml version="1.0" encoding="UTF-8"?>
2
3 <!-- This Listing provides the sample XML File that will be encrypted. -->
4 <purchaseOrder>
5   <Order>
6     <BookName>Soccer For Dummies</BookName>
7     <Id>123-958-74598</Id>
8     <Quantity>500</Quantity>
9   </Order>
10  <Payment>
11    <EncryptedData xmlns="http://www.w3.org/2001/04/xmlenc#" Id="Test"
12      Type="http://www.w3.org/2001/04/xmlenc#Content">
13      <EncryptionMethod Algorithm="http://www.w3.org/2001/04/xmlenc#tripledes-cbc" />
14      <ds:KeyInfo xmlns:ds="http://www.w3.org/2000/09/xmldsig#">
15        <KeyName>theKey</KeyName>
16      </ds:KeyInfo>
17      <CipherData>
18        <CipherValue>6n8juQbwTMrXB/3RNvvyul+tCLP1YxYf4m9XUNdwQT+fj7TFV2yW45YvALTycTHU
19        mmu8zj200iPb
20        47F/1Bf8lnhkvo0aVSLpKJsNx10oWv23sKnLyvbGsou2FgDGx0NgMubwgui3ALM=</CipherValue>
21      </CipherData>
22    </EncryptedData>
23  </Payment>
24 </purchaseOrder>
```

รูปที่ 4.15 แสดงการเข้ารหัส เฉพาะ Element < Payment > แบบ tripledes

4.2.2.3.3. การเข้ารหัสเฉพาะข้อมูลในเอกสาร

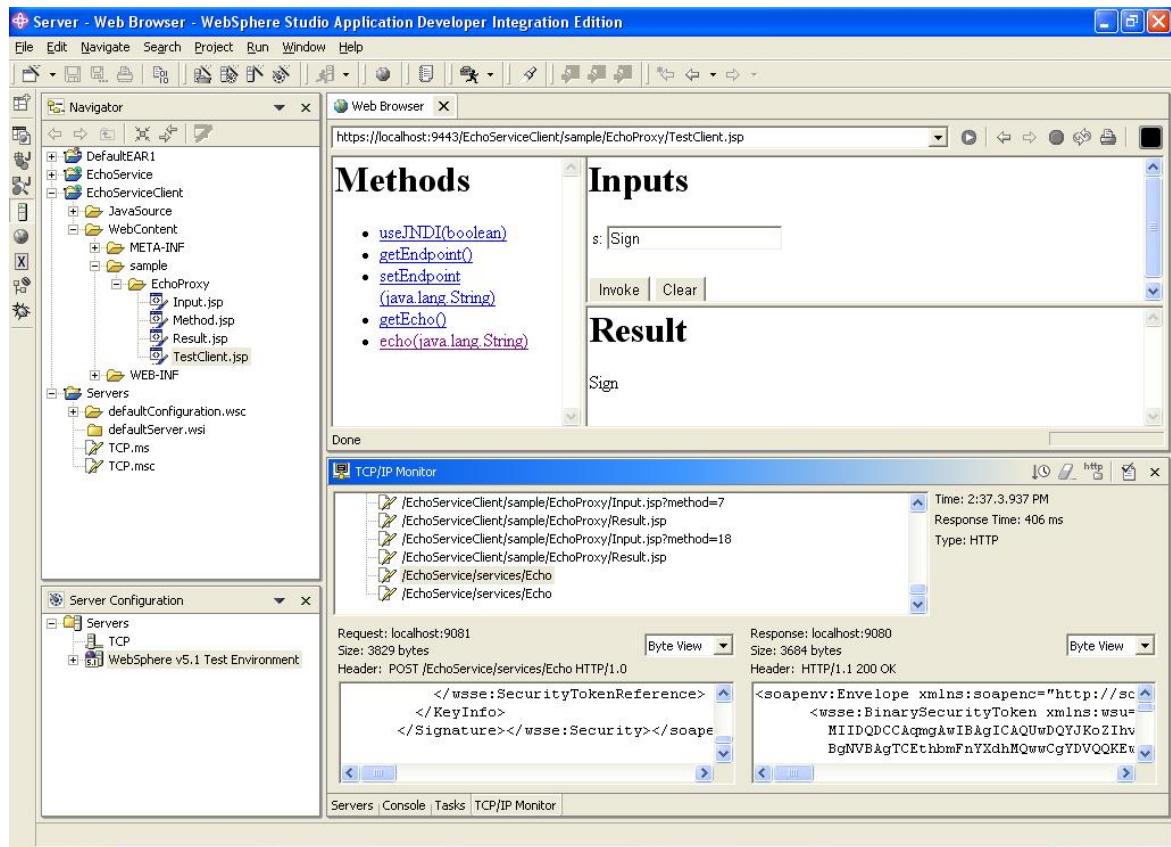
3.xml

10/3/2005 2:00 PM

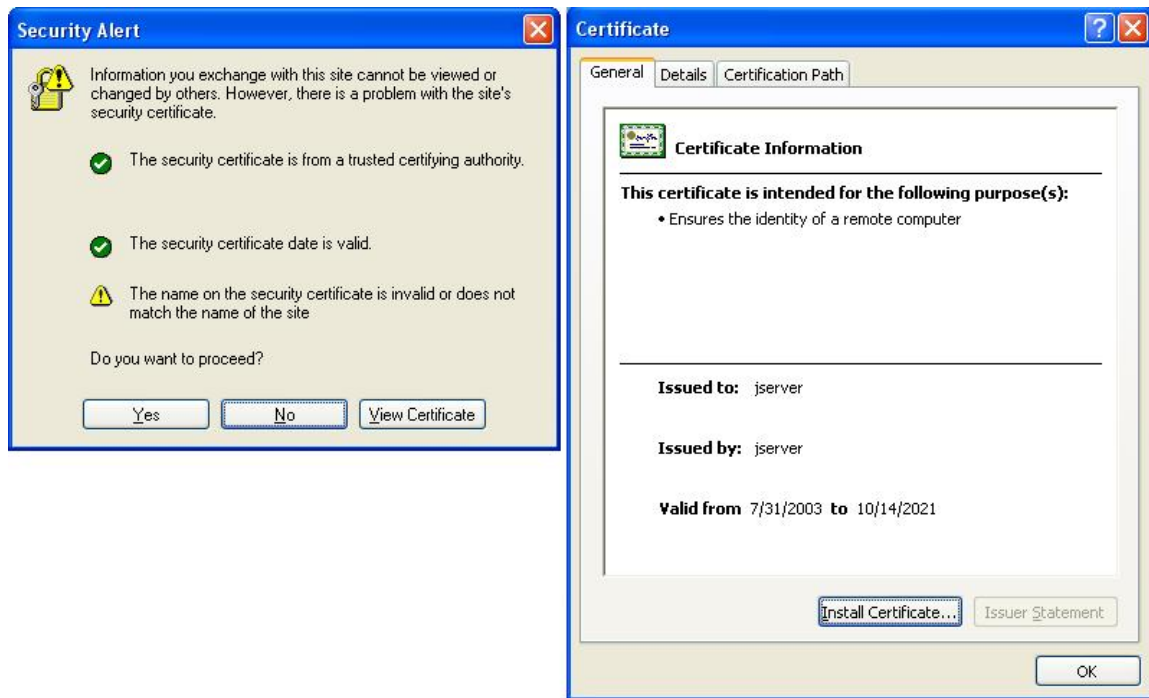
```
1 <?xml version="1.0" encoding="UTF-8"?>
2
3 <!-- This Listing provides the sample XML File that will be encrypted. -->
4 <purchaseOrder>
5   <Order>
6     <BookName>Soccer For Dummies</BookName>
7     <Id>123-958-74598</Id>
8     <Quantity>500</Quantity>
9   </Order>
10  <Payment>
11    <CardNo>
12      <EncryptedData xmlns="http://www.w3.org/2001/04/xmlenc#" Id="Test"
13        Type="http://www.w3.org/2001/04/xmlenc#Content">
14        <EncryptionMethod Algorithm="http://www.w3.org/2001/04/xmlenc#tripledes-cbc" />
15        <ds:KeyInfo xmlns:ds="http://www.w3.org/2000/09/xmldsig#">
16          <KeyName>theKey</KeyName>
17        </ds:KeyInfo>
18        <CipherData>
19          <CipherValue>d05qh+vPydTPfmrAuhkP208YeuD+N+JT56ALxj9eY=</CipherValue>
20        </CipherData>
21      </EncryptedData>
22    </CardNo>
23    <CardType>VISA</CardType>
24    <ValidDate>12-10-2004</ValidDate>
25  </Payment>
26 </purchaseOrder>
27
```

รูปที่ 4.16 แสดงการเข้ารหัสเฉพาะข้อมูลใน < CardNo > แบบ tripledes

4.2.2.4. การเพิ่มความปลอดภัยในการขนส่งข้อมูล (Transport – level Security) โดยการนำ SSL ไปใช้ในเว็บเซิร์ฟเวอร์



รูปที่ 4.17 แสดง Echo Service ที่มีการ Configure SSL ที่ พอร์ต 9443



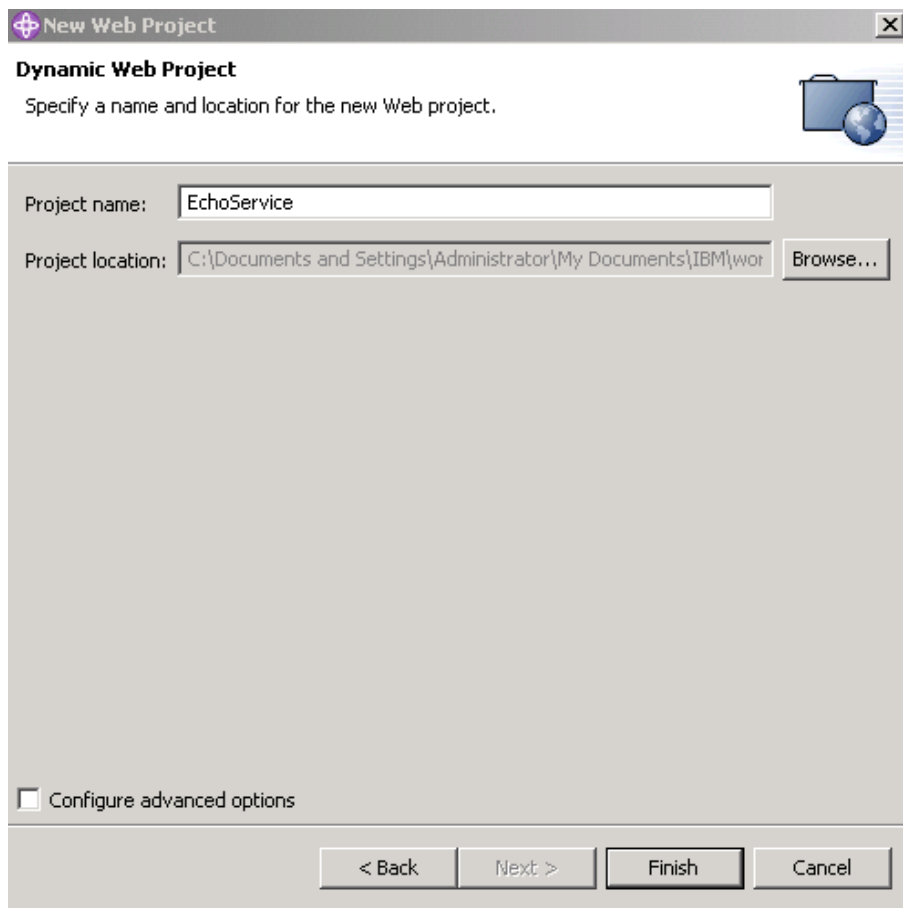
รูปที่ 4.18 แสดง Certificate และ แสดงว่า มีการติดตั้ง SSL บน Server เรียบร้อยแล้ว

4.2.3. การเซตค่าเพื่อใช้ในการทำ WS-Security แบบ Encryption ระหว่าง Websphere Application Developer 5.1 และ Microsoft Visual Studio .Net 2003

4.2.3.1 การเซตค่าของ Websphere Application Developer 5.1

4.2.3.1.1 การสร้างเว็บเซอร์วิส

1. สร้างเว็บเซอร์วิสโดยเลือกที่ File->New->Web->Dynamic Web Project จากนั้นทำการตั้งชื่อโปรเจกต์ ดังตัวอย่าง ในที่นี้ใช้ Echo Service



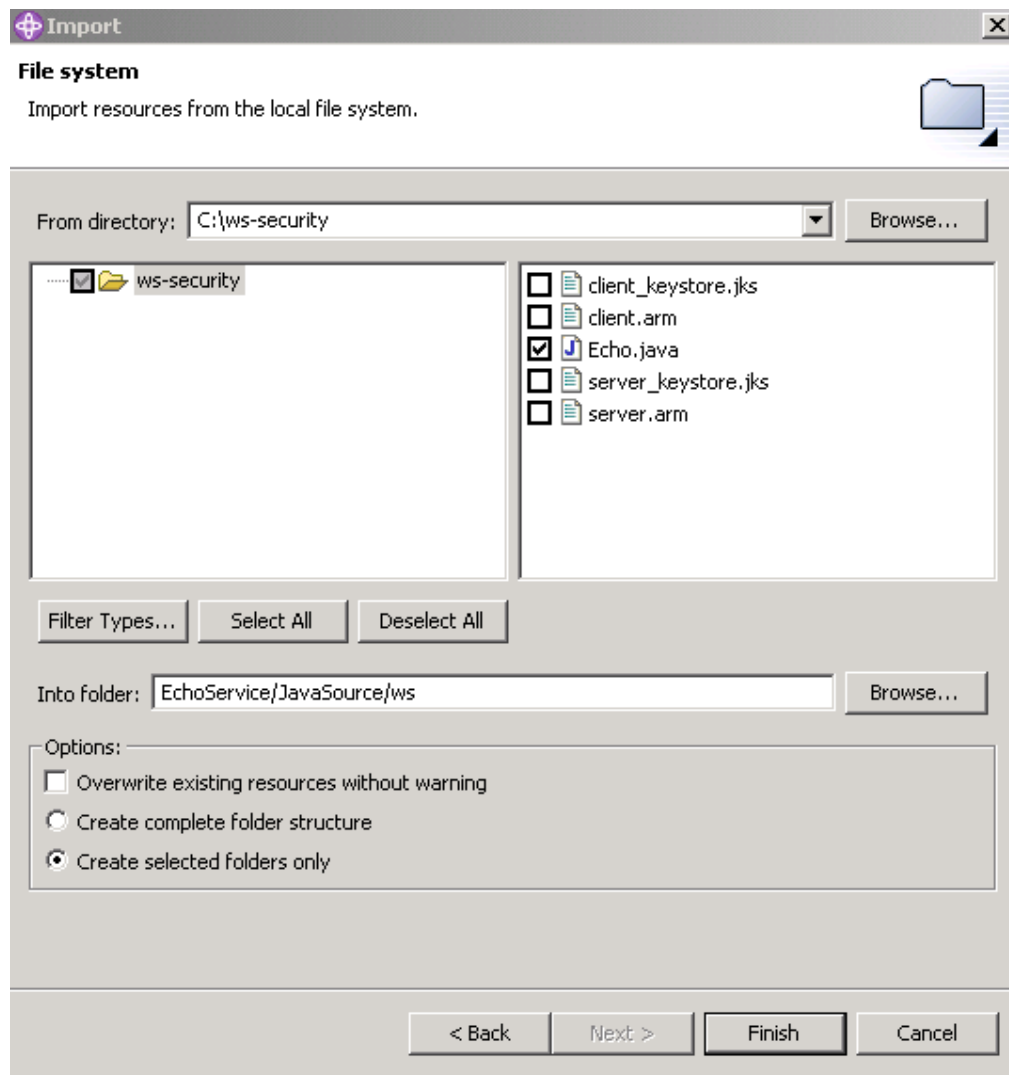
รูปที่ 4.19 แสดง การสร้างเว็บเซอร์วิส

2. เขียน Function ที่ต้องการ provide เป็นเว็บเซอร์วิส โดยการสร้างไฟล์จาวาขึ้นมาใหม่ File->new ->Java ->Class เช่น

```
package ws;

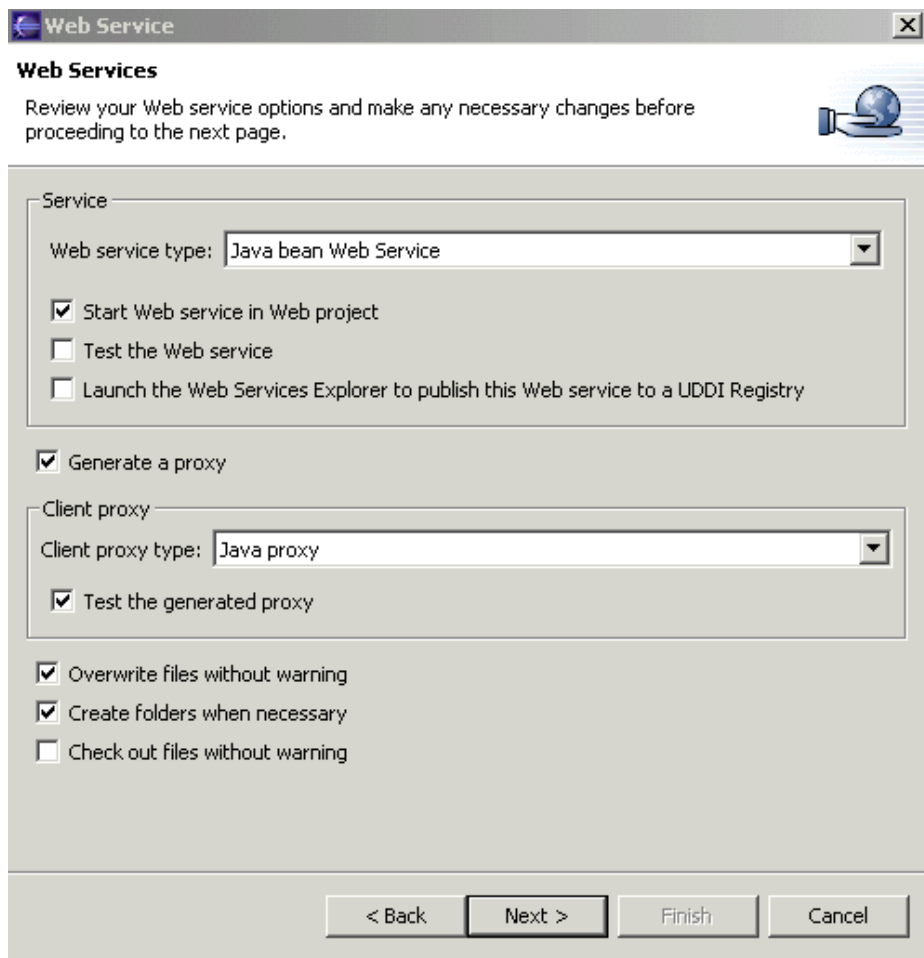
public class Echo
{
    public String echo(String s)
    {
        return s;
    }
}
```

หรืออาจทำได้โดยการ Import ไฟล์จาวาที่มีอยู่แล้วโดยเลือก File->import เช่น



รูปที่ 4.20 แสดงการ import ไฟล์จาวา เพื่อใช้ในการทำเว็บเซอร์วิส

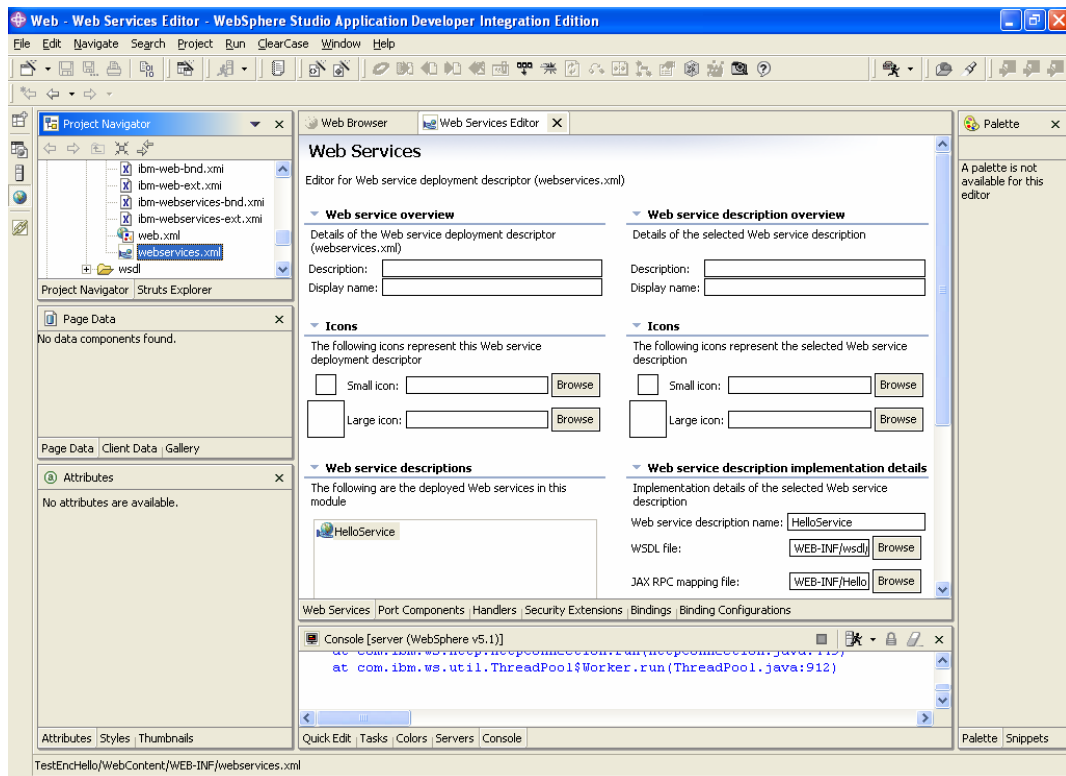
3. ทำการสร้างเว็บเซอร์วิสโดยเลือก File->New ->other->Web Service->Web Service เลือก. Java bean Web Service, Overwrite files without warning checkboxes. Next: -> finish



รูปที่ 4.21 แสดงการสร้างเว็บเซอร์วิส

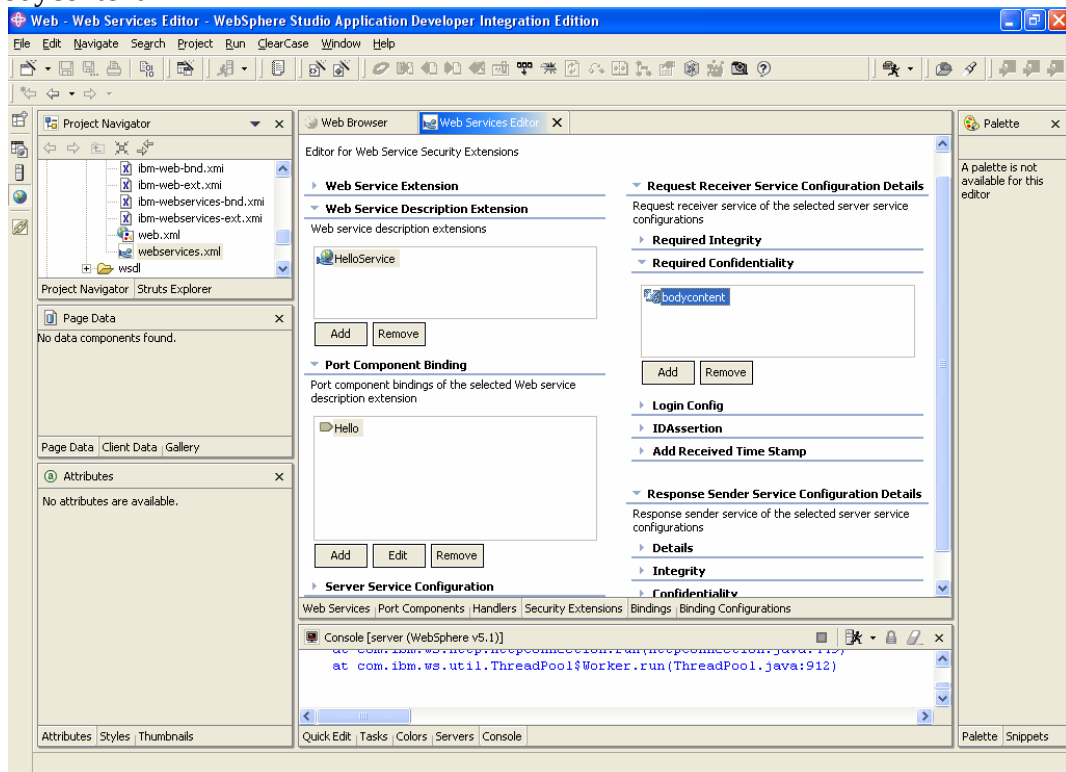
4.2.3.1.2. การ configure WS-Security แบบ Encryption บน Websphere

1. เข้าไปแก้ไขไฟล์ webservice.xml ซึ่งอยู่ใน directory -> WEB-INF -> webservice.xml



รูปที่ 4.22 แสดงข้อมูลในไฟล์ webservice.xml

2. เลือก Security Extension Tab ->Require Confidentiality ->Add -> bodycontent



รูปที่ 4.23 แสดงการConfigure ใน security extension

3. เลือก Binding Configurations TAB -> Request Receiver Binding Configuration-> Keylocator ระบุตามรูปด้านล่าง

Key locator dialog

Key locator name: Interops

Key locator class: com.ibm.wsspi.wssecurity.config.KeyStoreKeyLocator

☒ Use key store

Key store storepass: interops

Key store path: c:/Enc_key/interops-enc-receiver.jceks

Key store type: JCEKS

Key:

Alias	Key pass	Key name
interops-private-key	interops	interops

Add Remove

Property:

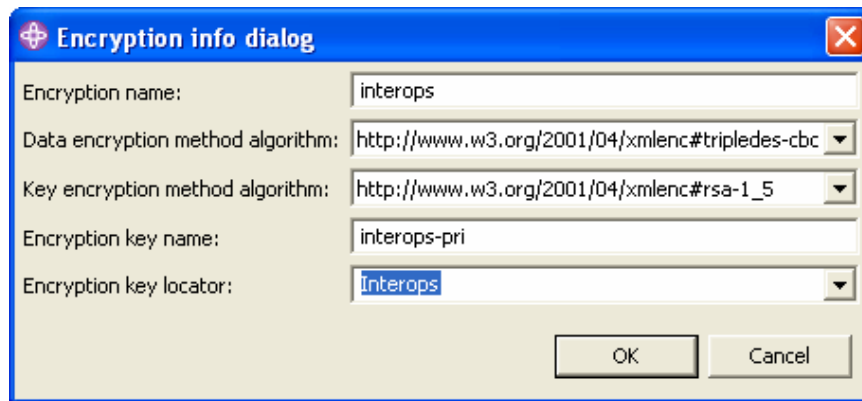
Name	Value

Add Remove

OK Cancel

รูปที่ 4.24 แสดงการ configure Key ที่ใช้ในการ Encryption

4. เลือก Binding Configurations TAB -> Request Receiver Binding Configuration-> Encryption และระบุตามรูป



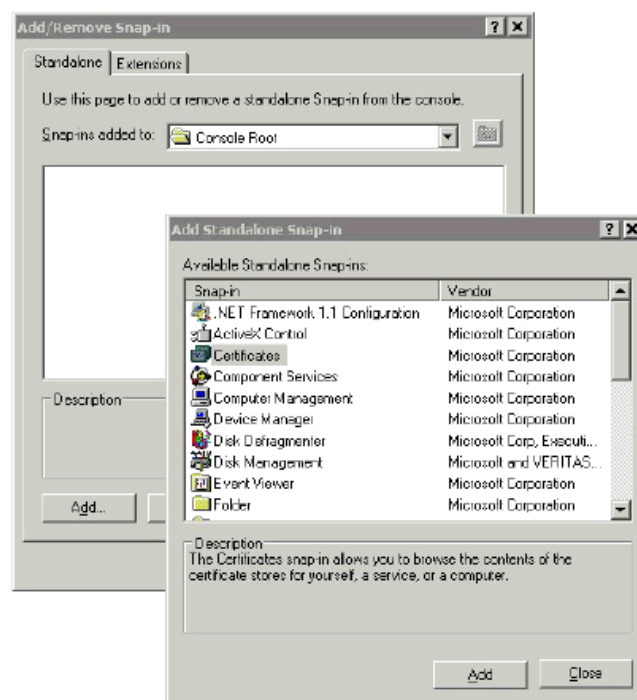
รูปที่ 4.25 แสดงการ configure encryption

จากนั้นกด Save และทำการ Restart Server และนี่ก็คือการ configure WS-Security บน Websphere

4.2.3.2 การเซตค่าของ Microsoft Visual Studio .Net 2003 with WSE 1.0

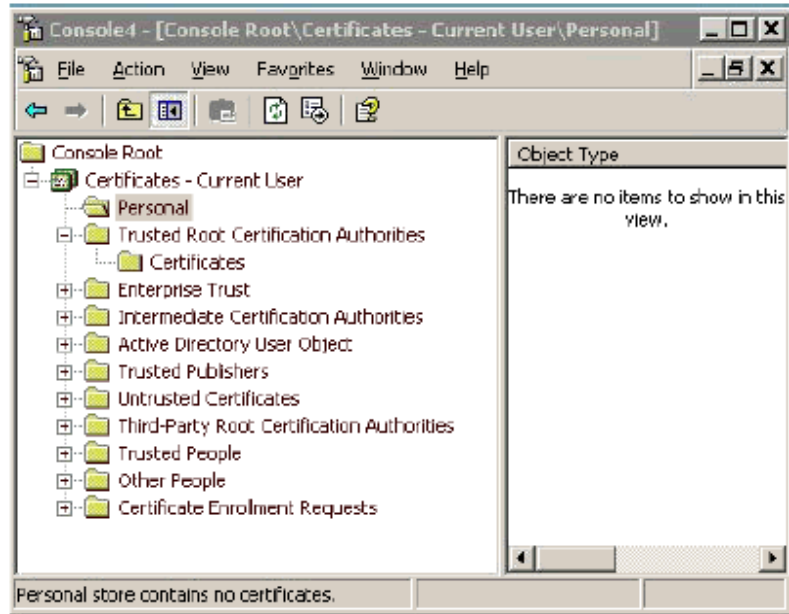
4.2.3.2.1 การ install key

1.ในการจะทำSecurity บน Microsoft .Net จะต้องมีการ Add certificate ใส่ใน store ซึ่งสามารถทำได้โดย Start Menu ->run พิมพ์ mmc จากนั้นจะเข้าสู่ console ให้เลือก Add/Remove Snap-in ซึ่งจะแสดงหน้าจอตามรูปด้านล่างขึ้นมา



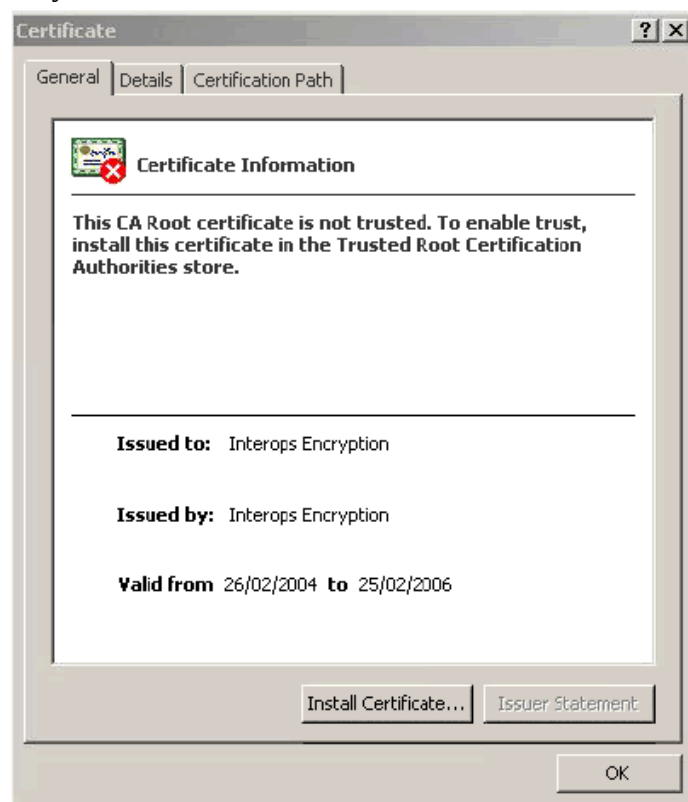
รูปที่ 4.26 แสดงการ เพิ่มคีย์

2.เลือก Certificate ->manage user account เมื่อเลือกแล้วจะแสดงหน้าต่างดังนี้



รูปที่ 4.27 แสดง Store ใน Windows

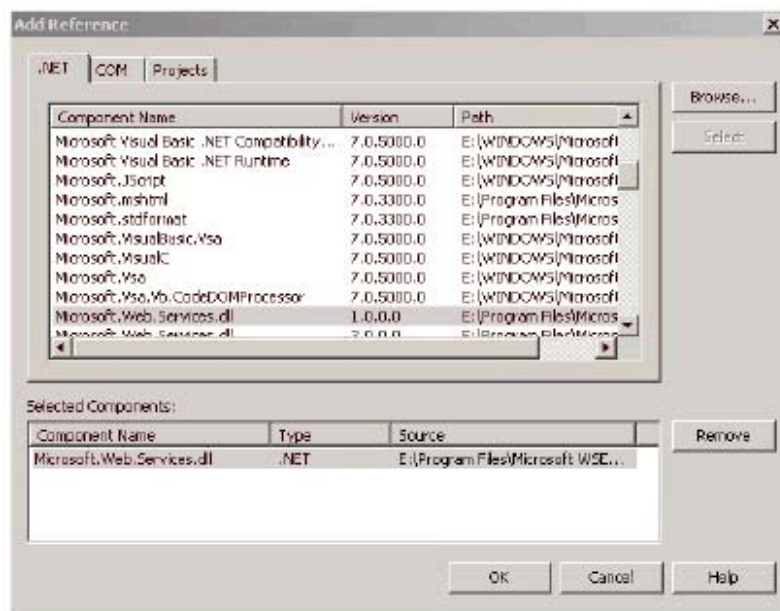
3. ต้องทำการ import public key เข้าไปใน Personal Store & Trusted Root Certificate Authority



รูปที่ 4.28 แสดง Certificate ที่ import เข้ามาแล้ว

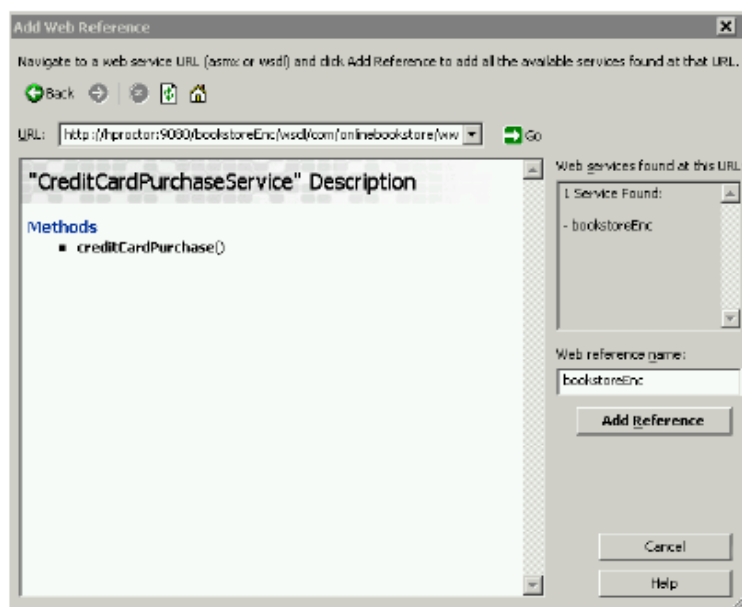
4.2.3.2.2. การสร้างเว็บแอปพลิเคชันของ ASP ด้วย Microsoft Visual Studio .Net 2003ที่มีการติดตั้ง WSE 1.0 และ IIS เรียบร้อยแล้ว

1. ทำการสร้างเว็บเซอร์วิส โดยเข้าไปใน Visual Studio .Net 2003 จากนั้น เลือก ASP .Net Web Application เมื่อสร้างเสร็จแล้วจึงทำการ Add Reference โดย Add Microsoft.Web.Services.dll ซึ่งเป็นไฟล์ของ WSE 1.0



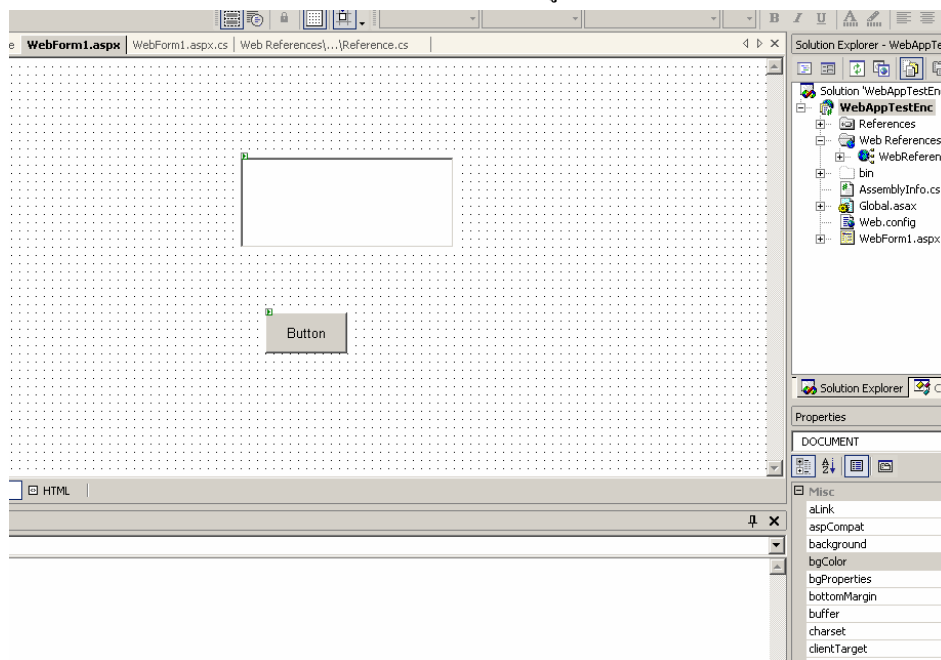
รูปที่ 4.29 แสดงการ Add reference Class ของ WSE 1.0

2. จากนั้นทำการ Add Web Reference ของ เว็บเซอร์วิส

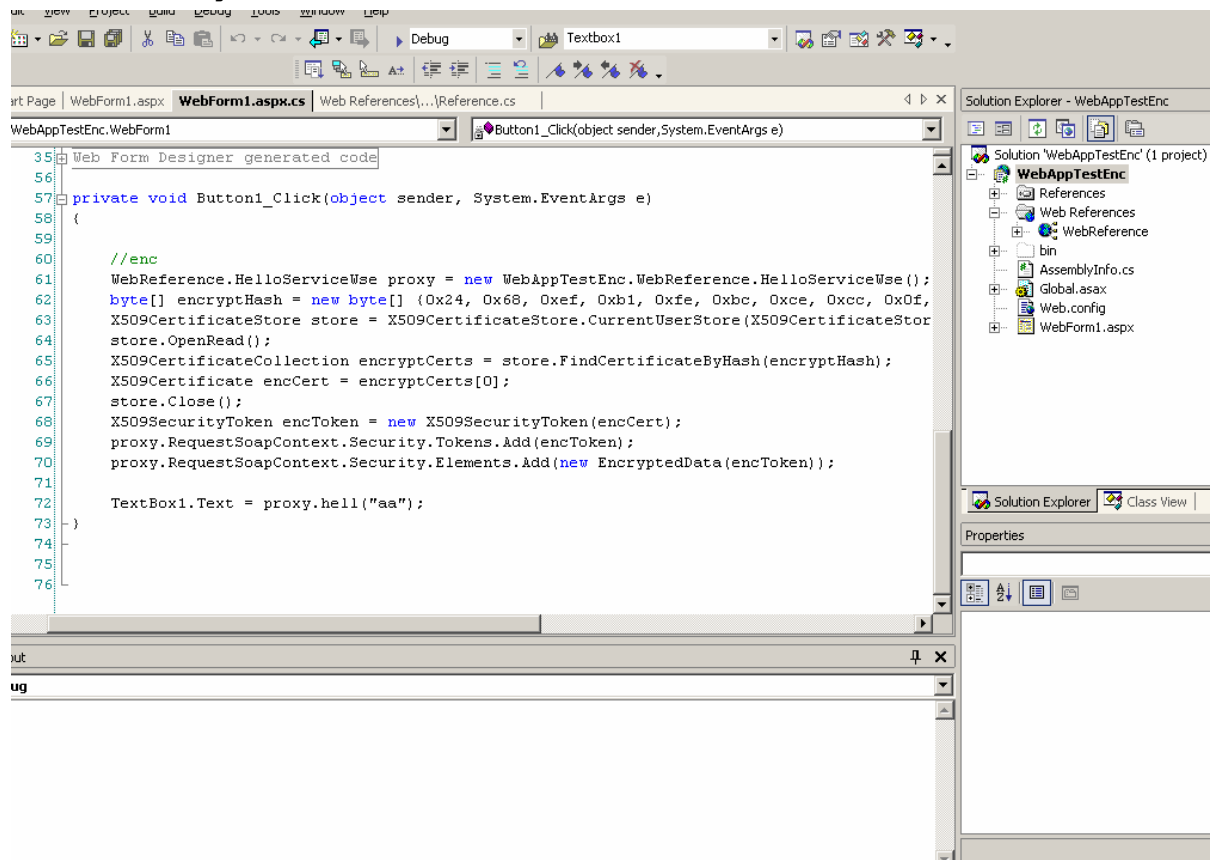


รูปที่ 4.30 แสดง Add Web reference ของ Web Service โดยระบุ WSDL

3. จากนั้นทำการ สร้างเว็บแล้ว เพิ่ม code ลงไปตามรูป



รูปที่ 4.31 ออกแบบหน้าตาของเว็บให้สอดคล้องกับเว็บเซอร์วิส



รูปที่ 4.32 เพิ่ม code ของโปรแกรม เพื่อใช้ในการ encryption

จากนั้นทำการ run program ก็จะได้ผลตามที่ต้องการ โดยมีการดักจับ SOAP Packet โดยใช้โปรแกรม Ethereal จะได้ผลดังนี้


```

3 <?xml version="1.0" encoding="utf-8"?>
4 <soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" xmlns:xsd="http://www.w3.org/2001/XMLSchema">
5 <soap:Header>
6 <wsrp:path soap:actor="http://schemas.xmlsoap.org/soap/actor/next" xmlns:wsrp="http://schemas.xmlsoap.org/rp">
7 <wsrp:action></wsrp:action><wsrp:to>http://161.246.6.91:9080/TestEncHello/services/Hello</wsrp:to>
8 <wsrp:id>uid:9d345c1d-0959-400c-8d3b-b38871d0b787</wsrp:id></wsrp:path>
9 </wsrp:Timestamp xmlns:wsu="http://schemas.xmlsoap.org/ws/2002/07/utility">
10 <wsu:Created>2006-02-16T15:49:27Z</wsu:Created><wsu:Expires>2006-02-16T15:54:27Z</wsu:Expires>
11 </wsu:Timestamp><wsse:Security soap:mustUnderstand="1" xmlns:wsse="http://schemas.xmlsoap.org/ws/2002/07/secext">
12 <wsse:BinarySecurityToken ValueType="wsse:X509v3" EncodingType="wsse:Base64Binary"
  xmlns:wsu="http://schemas.xmlsoap.org/ws/2002/07/utility"
  wsu:Id="SecurityToken-fe9540a0-19d7-43dd-8323-b11854dba8a3">MIICKTCCA2KGAwIBAgIBADANBqkqhkiG9w0BAQQFADAEMRwwGgYDVQQDE
  xNjbnRlcml9wcyBfBmNyeXB0aW9uMB4XDTA0MDIyNjIxMzcwNloXDTA2MDIyNTIxMzcwNloHjEcmBoGALUEAMTSW50ZXJvcHMGRW5jcmlwdGlvb1CBnz
  ANBqkqhkiG9w0BAQEFAAOBjQAwgYkCgYEAAn/gbFdD5XimDJiHoe3Gt8A6nZH+FnkSx//eBSf0+leoUIF4eloDBybR1qW8P1kKiTVWJALgJ5cYXZPASLk
  KPGARmdCwtF9Gw7JAXZ7wmEkqxTaHvVjwBIRfvjSWaRm0dCrXdcH2+XBPOMIqKIVZdX+G/k0kCEIEJeQw3b0pJOCaWEAAa3MHUwHQYDVRO0BBYEFF/3
  CTVOjzLD+WAtwo02QGPned+MEYGA1UdIwQ/MD2AFF/3CTVONjzLD+WAtwo02QGPned+oSKkIDAeMRwwGgYDVQQDExNjbnRlcml9wcyBfBmNyeXB0aW9uag
  gEAMAwGALUdEwQFMAMBAf8wDQYJKoZIhvcNAQEEBQADgYEAG8bZL2DfTc9M2F8V3BnC+OFuVHa7v7qjQAANTqoxbStjmFMo8WvCMMLqX01Xpxbh71sFjbS
  lIUwG72RxsmpsaIs/HOSN8JL0kUFFPhnlg1vAfpATHwr+UjNpMvffvpFo16K1DRTicELnC8Wtd4MYQ82DceKkd4FV9gP4X+YQDE4WI=
  </wsse:BinarySecurityToken><xenc:EncryptedKey Type="http://www.w3.org/2001/04/xmenc#EncryptedKey"
  xmlns:xenc="http://www.w3.org/2001/04/xmenc#"><xenc:EncryptionMethod
  Algorithm="http://www.w3.org/2001/04/xmenc#rsa-l_5" /><KeyInfo xmlns="http://www.w3.org/2000/09/xmldsig#"
  >
13 <wsse:SecurityTokenReference><wsse:KeyIdentifier
  ValueType="wsse:X509v3">X/cJMXQ2PMsP5YC3CjTZAAk1534=</wsse:KeyIdentifier></wsse:SecurityTokenReference></KeyInfo>
14 <xenc:CipherData><xenc:CipherValue>fG0va7DzVvG4ym8V0epG7oKaqvHiDVrLrQ1+FbwiV1ZSDdgFXDQCFCtdZw5egb3hoRVBRcatApS2ZNT1A
  MW0X8ozVS171lr29I/vEGbqPfln8BauXfojzkiSrZUrbpDHOD0BjqHFNqph5e3I8ve64t1GGliceQe9UvJLtna0yI=</xenc:CipherValue>
15 </xenc:CipherData>
16 <xenc:ReferenceList><xenc:DataReference URI="#EncryptedContent-bbd53227-ea32-446e-85d4-fa05cca20654" />
17 </xenc:ReferenceList></xenc:EncryptedKey></wsse:Security></soap:Header>
18 <soap:Body xmlns:wsu="http://schemas.xmlsoap.org/ws/2002/07/utility"
  wsu:Id="Id-6b89cef0-b2c3-4174-ae97-aa5da7078ce4">
19 <xenc:EncryptedData Id="EncryptedContent-bbd53227-ea32-446e-85d4-fa05cca20654"
  Type="http://www.w3.org/2001/04/xmenc#Content" xmlns:xenc="http://www.w3.org/2001/04/xmenc#">
20 <xenc:EncryptionMethod Algorithm="http://www.w3.org/2001/04/xmenc#tripleDES-cbc" />
21 <xenc:CipherData><xenc:CipherValue>owCUN7ipzPIR2nrm7i9vxh3qUcA3ZvGK30VChSCLw5xa5//sW9nD09b+BdxnYnkSzcewnck0FWI=
22 </xenc:CipherValue></xenc:CipherData></xenc:EncryptedData></soap:Body></soap:Envelope>

```

รูปที่ 4.33 แสดง SOAP Message ที่มีการใช้ WS-Security

4.2.4. การเซตค่าเพื่อใช้ในการทำ WS-Security แบบ Digitalsignature ระหว่าง Websphere

Application Developer 5.1 และ Microsoft Visual Studio .Net 2003

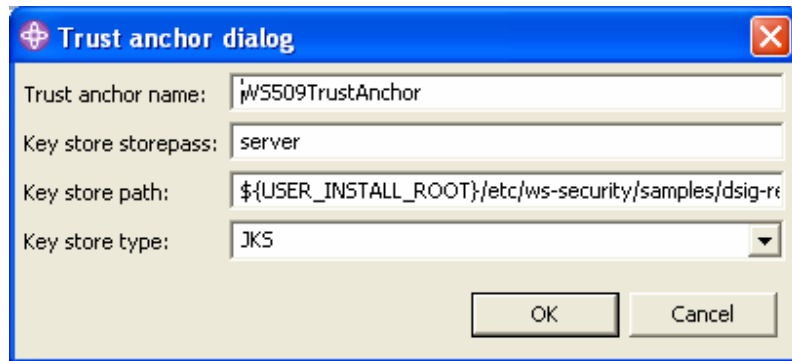
4.2.4.1. การเซตค่าของ Websphere Application Developer 5.1

- เมื่อสร้างเว็บเซอร์วิสเสร็จแล้วให้ไปconfigure webservice.xml ->security Extension ->request Receiver Configuration ->require integrity->body



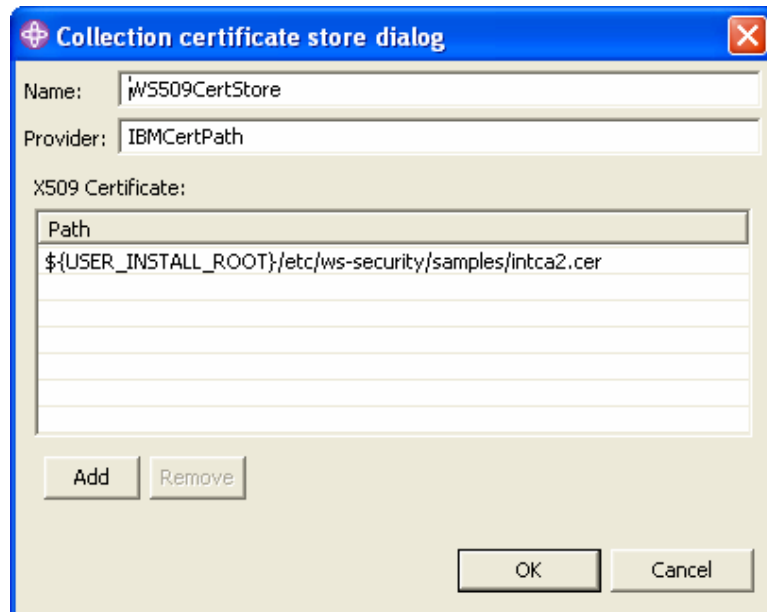
รูปที่ 4.34 แสดง require integrity

2. จากนั้นจึง Configure Binding configurations -> request receiver -> Trust Anchor เพื่อระบุคีย์ที่ต้องใช้ในการทำ signature



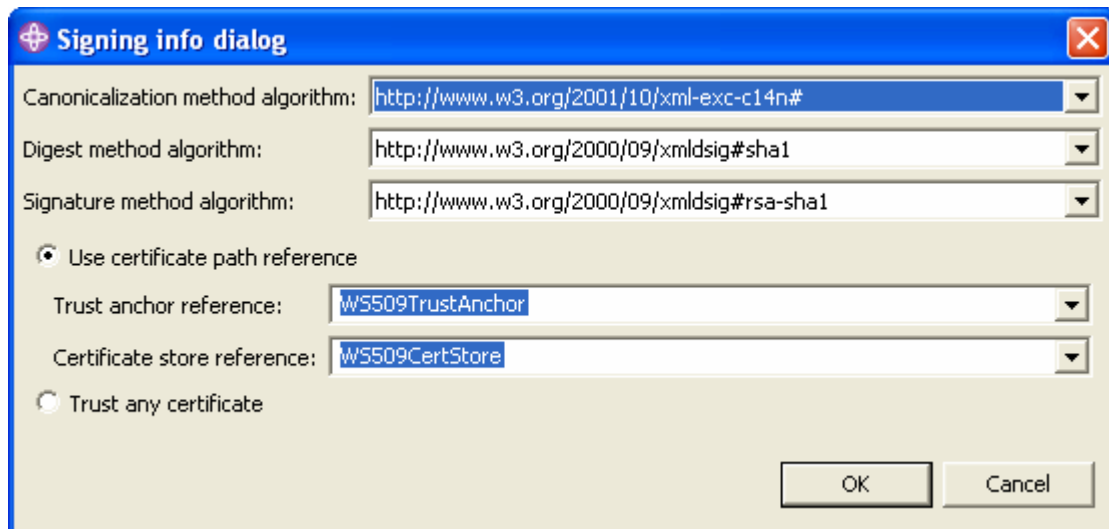
รูปที่ 4.35 แสดงการระบุคีย์ที่ต้องการใช้ใน signature

3. แล้วต่อมาจึง configure ส่วนของ certificate ใน certificate store list -> certificate store



รูปที่ 4.36 แสดง certificate ที่ใช้ trust

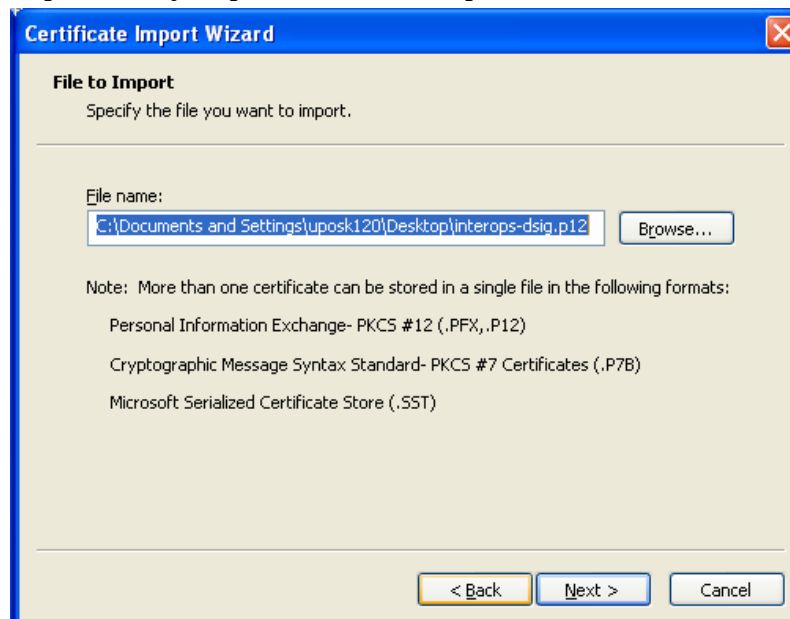
4. จากนั้นจึงไป configure เกี่ยวกับคีย์ใน Binding configurations -> request receiver -> signing information



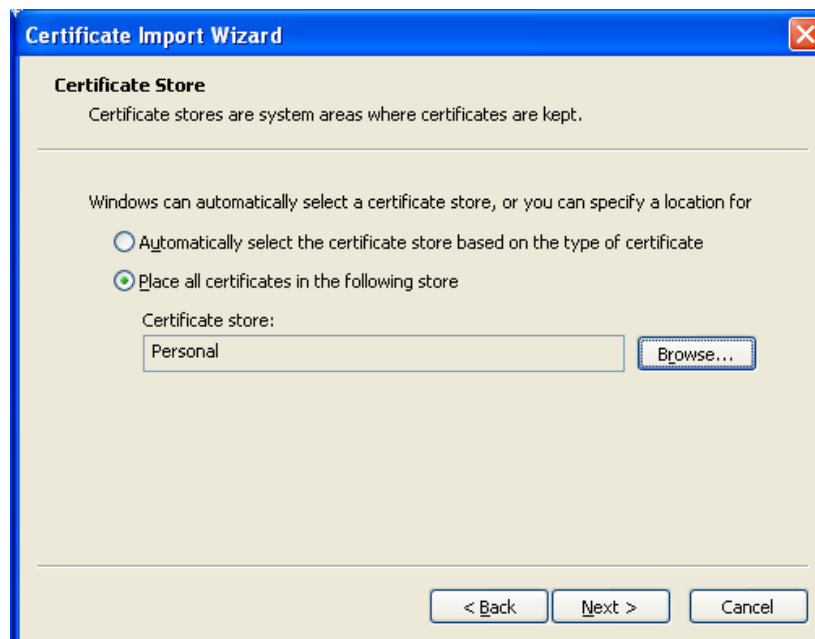
รูปที่ 4.37 แสดง การ configure ใน signing information

4.2.4.2. การเซตค่าของ Visual Studio .Net 2003

1. ต้องมีการ Add key สำหรับ Sign เพิ่มใน store โดยในที่นี้ ต้อง configure ในรูปแบบที่คล้ายๆกัน โดย ต้อง เพิ่ม คีย์ทั้ง 2 อย่าง คือ key ที่ใช้ในการ sign และ key ที่ใช้ในการ trust โดยในที่นี้ใช้ คีย์ interops-root.cer ซึ่งเป็น public key ใน trust และ personal และ ใช้ คีย์ interops-dsig.p12 ซึ่งเป็น private key ใน personal โดยการ import เข้าไปเหมือนเดิม



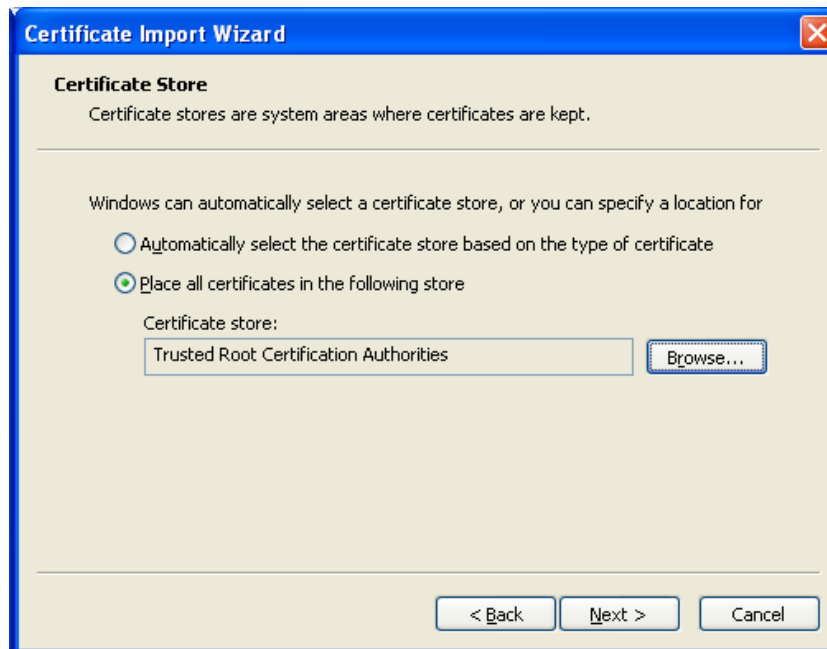
รูปที่ 4.38 แสดงการ import key interops-dsig.p12



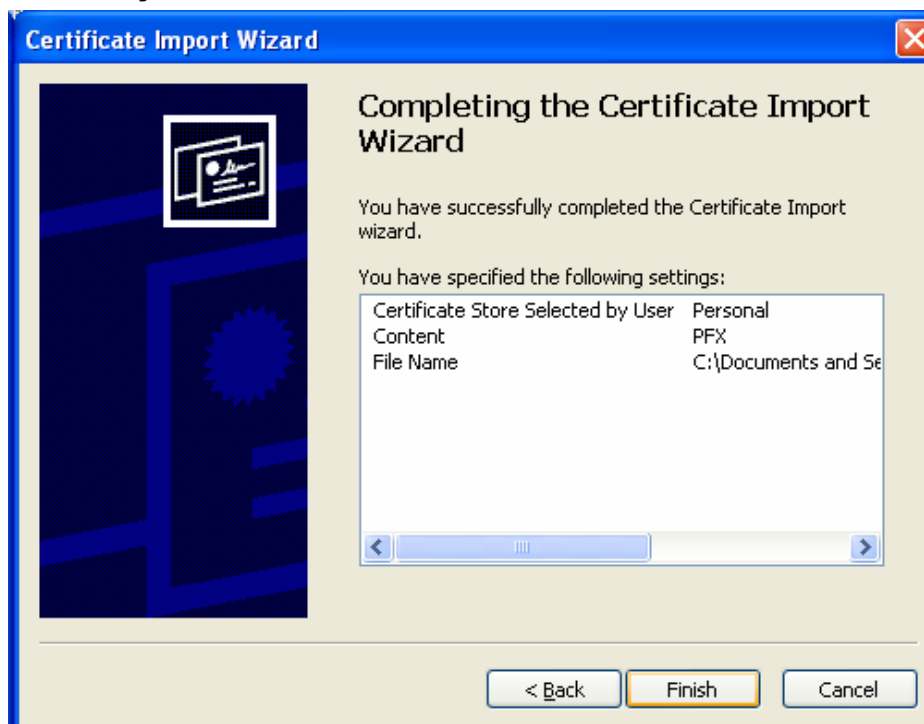
รูปที่ 4.39 แสดงผลการ import key ใน personal store



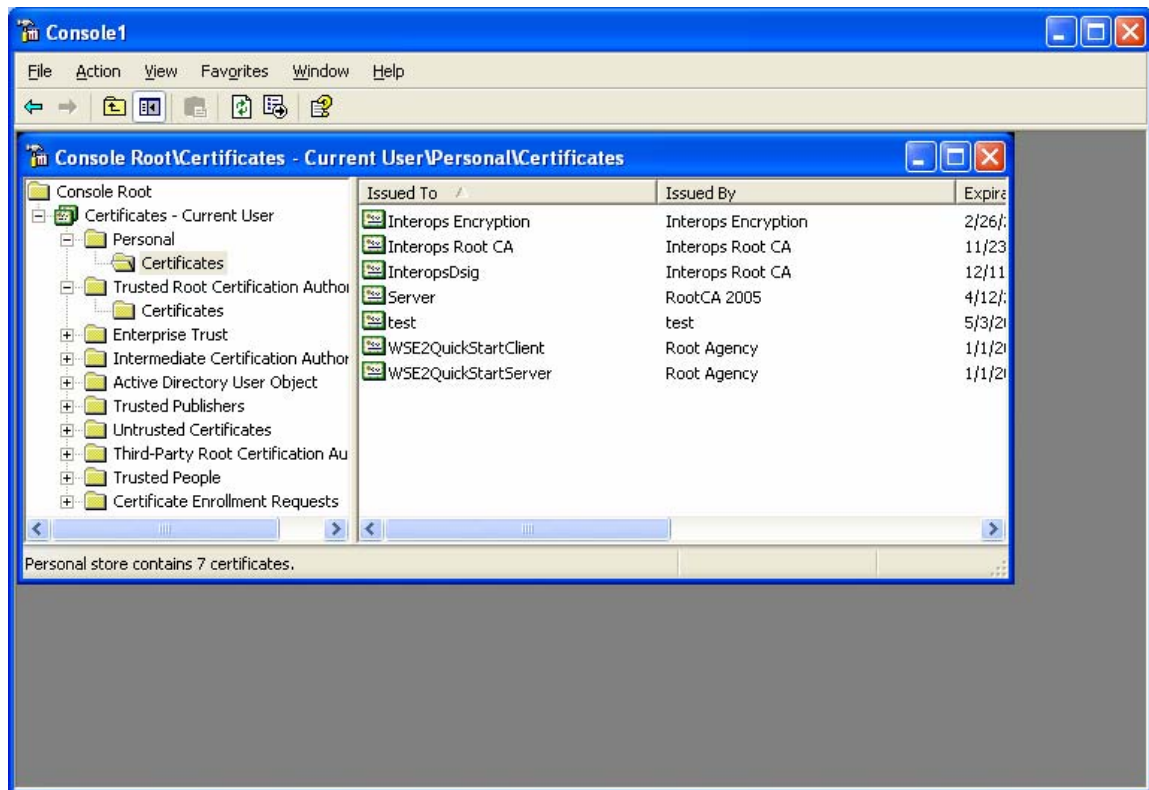
รูปที่ 4.40 แสดงการ import key pfx ที่มี private key ต้องมีการใส่รหัสผ่านด้วย



รูปที่ 4.41 ทั้ง 2 ก็ยัต้องทำให้ใน personal Store & Trust Root



รูปที่ 4.42 แสดงผลการ import ทั้ง 2 สำเร็จ



รูปที่ 4.43 แสดง key และ certificate ที่ import เข้าไปแล้ว

2. ต้องมีการ เพิ่ม code เข้าไปเพื่อให้สามารถใช้digital signature ได้ ใน Microsoft .net ดังนี้

```
private void Button1_Click(object sender, System.EventArgs e)
{
    encRef.HelloServiceWse proxy = new WebAppTestCer2.encRef.HelloServiceWse();
    byte[] encryptHash = new byte[] { 0x24, 0x68, 0xef, 0xb1, 0xfe, 0xbc, 0xce, 0xcc, 0x0f, 0x0f };
    byte[] signHash = new byte[] { 0x60, 0xac, 0x1e, 0x05, 0x5c, 0x84, 0xeb, 0xf2, 0x92, 0x0f };
    X509CertificateStore store = X509CertificateStore.LocalMachineStore(X509CertificateStore.OpenRead());
    X509CertificateCollection encryptCerts = store.FindCertificateByHash(encryptHash);
    X509CertificateCollection singCerts = store.FindCertificateByHash(signHash);
    X509Certificate encCert = encryptCerts[0];
    X509Certificate signCert = singCerts[0];

    X509SecurityToken encToken = new X509SecurityToken(encCert);
    X509SecurityToken signToken = new X509SecurityToken(signCert);

    proxy.RequestSoapContext.Security.Tokens.Add(encToken);
    proxy.RequestSoapContext.Security.Tokens.Add(signToken);
    proxy.RequestSoapContext.Security.Elements.Add(new EncryptedData(encToken));
    proxy.RequestSoapContext.Security.Elements.Add(new MessageSignature(signToken));
    store.Close();

    TextBox1.Text = proxy.hell();
}
```

รูปที่ 4.44 แสดง code ที่เพิ่มเข้าไปให้สามารถทำได้ ทั้ง Digital signature & Encryption


```

1 <soapenv:Envelope xmlns:soapenc="http://schemas.xmlsoap.org/soap/encoding/"
  xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/" xmlns:xsd="http://www.w3.org/2001/XMLSchema"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"><soapenv:Header><wsse:Security
  xmlns:wsse="http://schemas.xmlsoap.org/ws/2003/06/secext" soapenv:mustUnderstand="1">
2   <EncryptedKey xmlns="http://www.w3.org/2001/04/xmlenc#">
3   <EncryptionMethod Algorithm="http://www.w3.org/2001/04/xmlenc#rsa-1_5"></EncryptionMethod>
4   <KeyInfo xmlns="http://www.w3.org/2000/09/xmldsig#">
5     <wsse:SecurityTokenReference>
6       <wsse:KeyIdentifier>X/cJNXQ2PMsP5YC3CjTZAak1534=</wsse:KeyIdentifier>
7     </wsse:SecurityTokenReference>
8   </KeyInfo>
9   <CipherData>
10    <CipherValue>n40IN6m/fKuK6RRIP6AepDFs+ZO2v0vPTYrj7PLCv3VgfnvteKqwx7fNDWwGqN/MUVtKM+I
      ySOVB5WVQ7vA+7E69j6DP7nYiZHyCBe2ZhtSfYRT08MbJRCw7jH59+mqQB7OJyDjzIQ7dAxmuHjar7R9qN8p2d3M/H5gAhns=</CipherValue>
11    </CipherData>
12    <ReferenceList>
13      <DataReference URI="#wssecurity_encryption_id_7195244911905120095"></DataReference>
14    </ReferenceList>
15  </EncryptedKey>
16  <wsse:BinarySecurityToken xmlns:wsu="http://schemas.xmlsoap.org/ws/2003/06/utility" EncodingType="wsse:Base64Binary"
    ValueType="wsse:X509v3" wsuId="wssecurity_binary_security_token_id_5837437789421640883">
17    MIIDQTCCAqggAwIBAgICAQQAQwDQYJKoZIhvcNAQEFBQAwTjELMAkGA1UEBhMCSEAETAP
18    BgNVBAGTCEthbmFnYXdhMQwwCgYDVQQKEwNJKk0xDDAKBgNVBAsTA1RSTDEQMA4GA1UE
19    AxMHSW50IENBMjAeFw0wMTEwMDEwOTU0MDZaFw0xMTEwMDEwOTU0MDZaMFQxCzAJBgNV
20    BAYTAkpQMREwDwYDVQQIEWhLYW5hZ2F3YTEuMDEwOTU0MDZaMFQxCzAJBgNVBAsTA1RSTDEQMA4GA1UE
21    BgNVBAGTCEthbmFnYXdhMQwwCgYDVQQKEwNJKk0xDDAKBgNVBAsTA1RSTDEQMA4GA1UE
22    AaGBAMy3PZ1mPhrEsBvYiOulIPV3Uis5Yy6hmox2YwYc2nNDBPzKsIWUj/Q+fk+DNdY
23    6KEHmuDrcVcEma48J9X1a5avRlksQIKptKoVn4eBys2i/wkwyZqHdaFji79/MvnTRW8E
24    Vy99FNKw4PFnhOoe1tDcNBulH/fuGOz9EITV+fAgMBAAAGggEmMIBIJAjBgNVHRME
25    AjaANAsGA1UdDwQEAwIF4DAsBgglghkgBhvhCAQ0EHzYdTB3BIBINTTCBHZW51cmF0ZWQg
26    Q2YydgGmaWNhdGUwHQYDVR0OBBYEFiW3FD1cXie4j4zw1gAp4cuOAZ4IMIG6BgNVHSMF
27    gblwga+AFL35INU4+WRY09vaf9zOsP7QvO9voYGSplGPMIGMMQswCQYDVQQGEwJKUDER
28    MA8GA1UECBMIS2FuYWdhZ2ExDzANBgNVBACTBllhbWV0bzEMMAoGA1UEChMDSUJNMQww
29    CgYDVQQLEwNUUkwxGTAXBgNVBAMTEFNQVAgMi4xZFRlc3QgQ0ExIjAgBgkqhkiG9w0B
30    CQEWE21hcnV5YyW1hQGPwLmlibS5jb22CAGEBMA0GCSqGSIb3DQEBBQUAA4GBAHkthdGD
31    gCvdlL9/vXUo74xpfOQd/r1owBmMdb1TWdOyZwbOHC7IkUlnKrkI7SofwSLSDUP571i
32    iMXUx3tRdmAVCoDMMFuDxh9V7212luXccx0s1S5KN0D3xW97LLNegQC0/b+aFD8Xkw2U
33    5ZtwnbFTRgs097dmz09RosDKKLIM
34  </wsse:BinarySecurityToken>
35  <Signature xmlns="http://www.w3.org/2000/09/xmldsig#">
36    <SignedInfo>
37      <CanonicalizationMethod Algorithm="http://www.w3.org/2001/10/xml-exc-c14n#"></CanonicalizationMethod>
38      <SignatureMethod Algorithm="http://www.w3.org/2000/09/xmldsig#rsa-sha1"></SignatureMethod>
39      <Reference URI="#wssecurity_body_id_3707081873083827552">
40        <Transforms>
41          <Transform Algorithm="http://www.w3.org/2001/10/xml-exc-c14n#">
42            <ec:InclusiveNamespaces xmlns:ec="http://www.w3.org/2001/10/xml-exc-c14n#" PrefixList="xsi soapenc xsd #default wsu soapenv"
              "></ec:InclusiveNamespaces>
43          </Transform>
44        </Transforms>
45        <DigestMethod Algorithm="http://www.w3.org/2000/09/xmldsig#sha1"></DigestMethod>
46        <DigestValue>j3f4HlgszYrf7teVW37Jot08mc=</DigestValue>
47      </Reference>
48    </SignedInfo>
49    <SignatureValue>
50      Ys1AczkU52JE7J/DxFTuu0VXBxEhcQI6hsh9eFpeQNrV6QdJuHMBY0r2jQFO
51      zyrWTWghqQ8F4xcYjlyHg8dsIWDcHp8JWW5Lc/RyUavJ6vAzPj7NdCD/CXsG
52      TZuqh9J4XbGzJeTAojoYbpSG0FX1UgnPCjJqkOsgsisjJIF610=
53    </SignatureValue>
54    <KeyInfo>
55      <wsse:SecurityTokenReference>
56        <wsse:Reference URI="#wssecurity_binary_security_token_id_5837437789421640883"></wsse:Reference>
57      </wsse:SecurityTokenReference>
58    </KeyInfo>
59  </Signature></wsse:Security></soapenv:Header><soapenv:Body xmlns:wsu="http://schemas.xmlsoap.org/ws/2003/06/utility"
    wsuId="wssecurity_body_id_3707081873083827552"><EncryptedData xmlns="http://www.w3.org/2001/04/xmlenc#"
    Id="wssecurity_encryption_id_7195244911905120095" Type="http://www.w3.org/2001/04/xmlenc#Content"><EncryptionMethod
    Algorithm="http://www.w3.org/2001/04/xmlenc#tripleDES-cbc">
60  </EncryptionMethod>
61  <CipherData>
62    <CipherValue>9VrQiAHaqYT7fWIDoSzWqGpitxiB5al7VgnQtrmN7IAsamPJSbs4TQ==</CipherValue>
63  </CipherData>
64  </EncryptedData>
65 </soapenv:Body></soapenv:Envelope>
66

```

Digital Signature

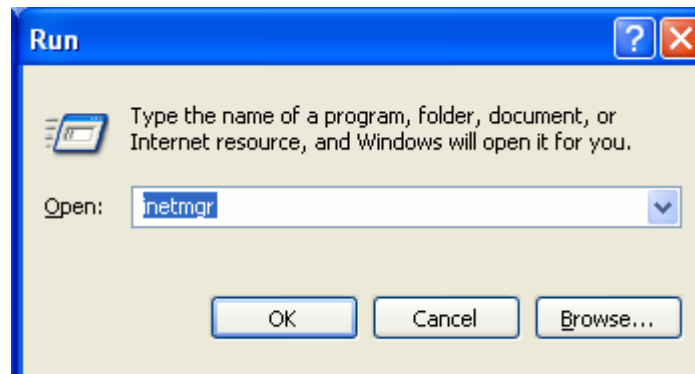
Data

รูปที่ 4.45 แสดง SOAP Message Encryption and Digital Signature

4.2.5. SSL

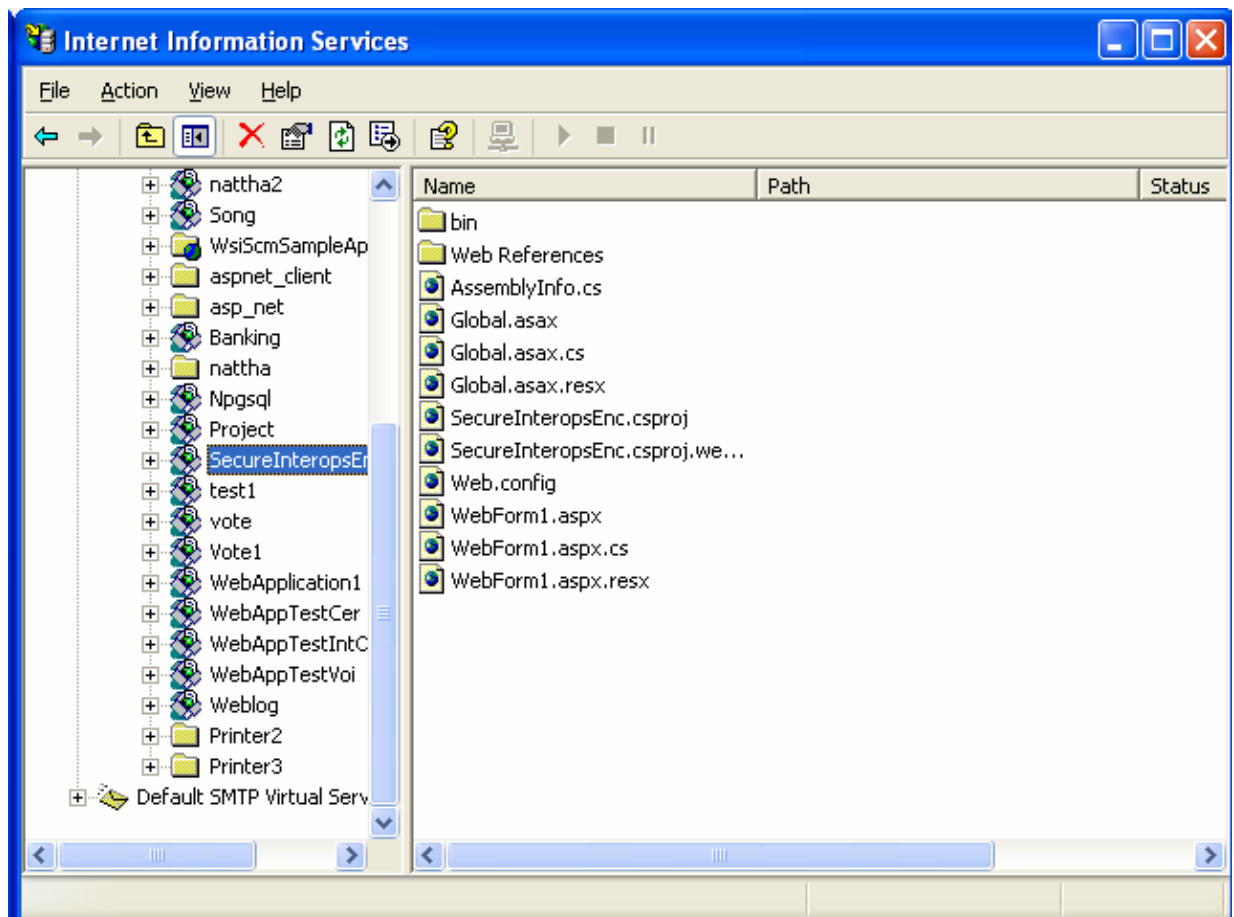
4.2.5.1 การติดตั้ง SSL บน IIS สำหรับใช้ใน ASP.Net Web Application

1. จัดการในส่วนของ IIS Manager โดย พิมพ์ inetmgr ที่ run...



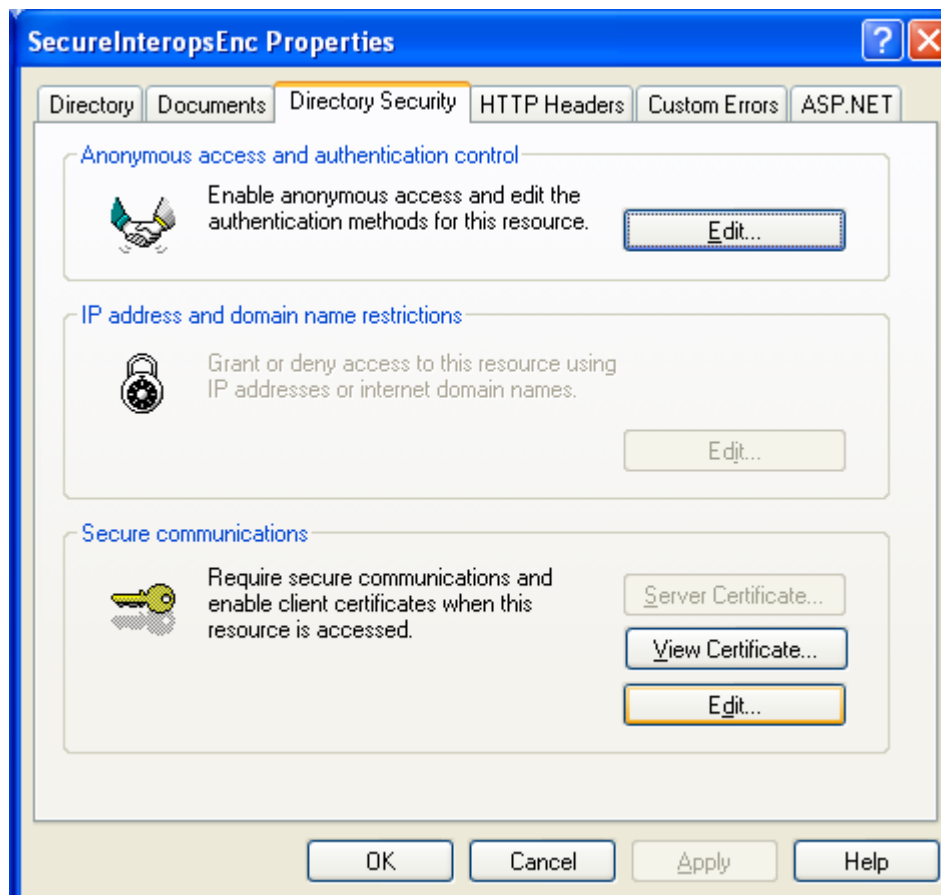
รูปที่ 4.46 แสดง การเข้าไปจัดการใน IIS Manager

2. เลือกเว็บไซต์ที่ต้องการทำ SSL



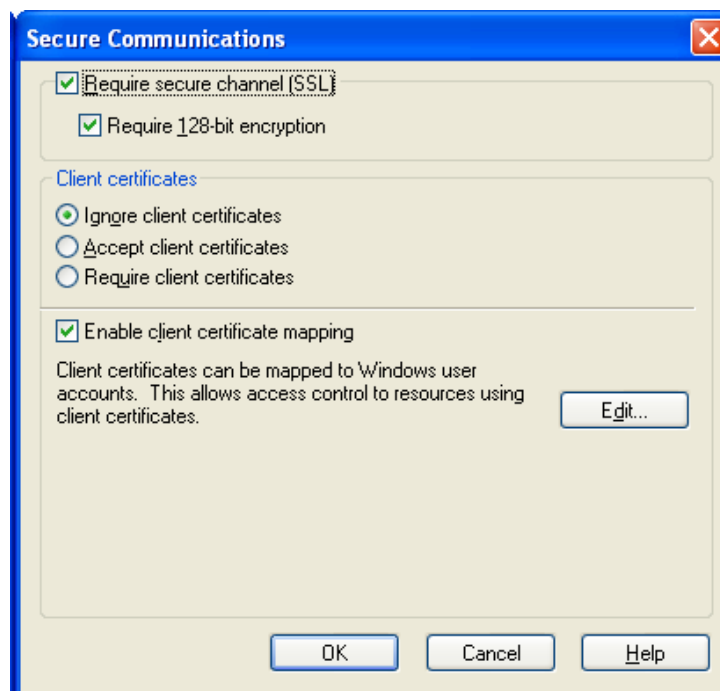
รูปที่ 4.47 แสดงการเลือกเว็บไซต์

3. เมื่อเลือกเว็บไซต์ได้แล้ว ให้ทำการ Configure secure communication โดยเลือก Edit

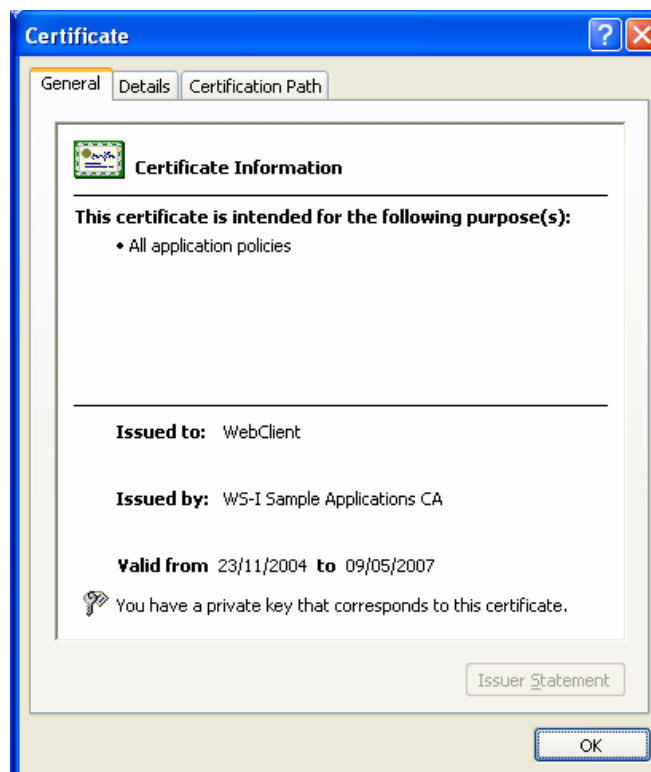


รูปที่ 4.48 แสดง properties ในเว็บต่างๆ

4. เลือก Require SSL /Require 128-bits encryption / Enable client certificate mapping

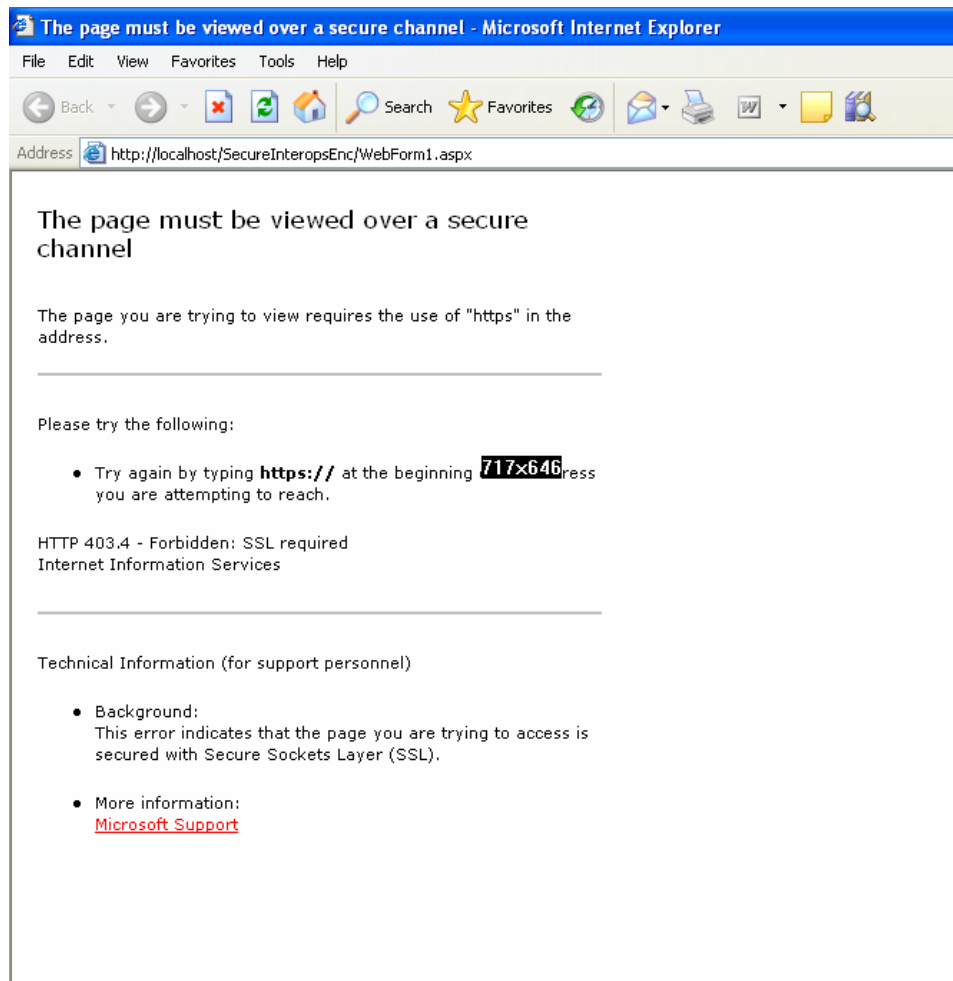


รูปที่ 4.49 แสดงการ ติดตั้ง SSL

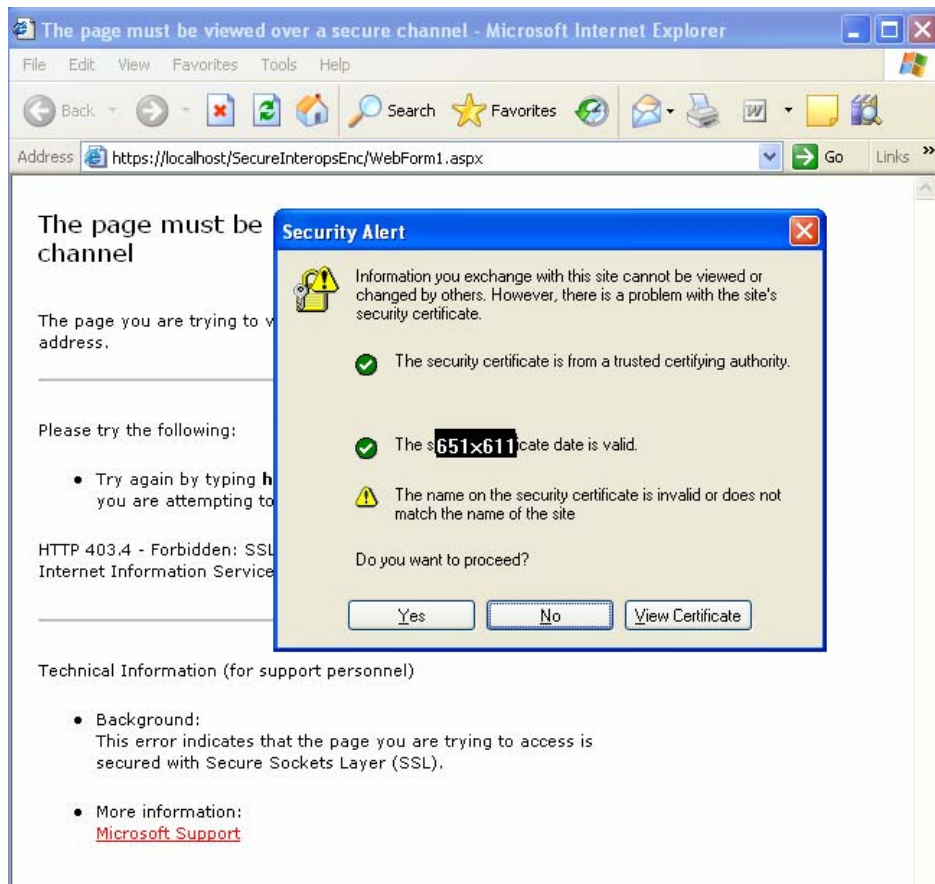


รูปที่ 4.50 แสดง certification ที่จะนำไปใช้ในการ SSL

เมื่อมีการ Configure SSL ในเว็บแล้ว จะไม่สามารถเรียกใช้เว็บผ่าน http protocol ได้ ต้องเรียกใช้งานในรูปแบบของ SSL เท่านั้น คือ https protocol จากนั้น จะมี ข้อความเตือนเรื่อง certificate เมื่อกดตกลงแล้ว จึงเป็นการใช้ SSL



รูปที่ 4.51 แสดงให้เห็นว่าเมื่อมีการ configure SSL แล้วจะไม่สามารถเข้าเว็บผ่าน http protocol ได้



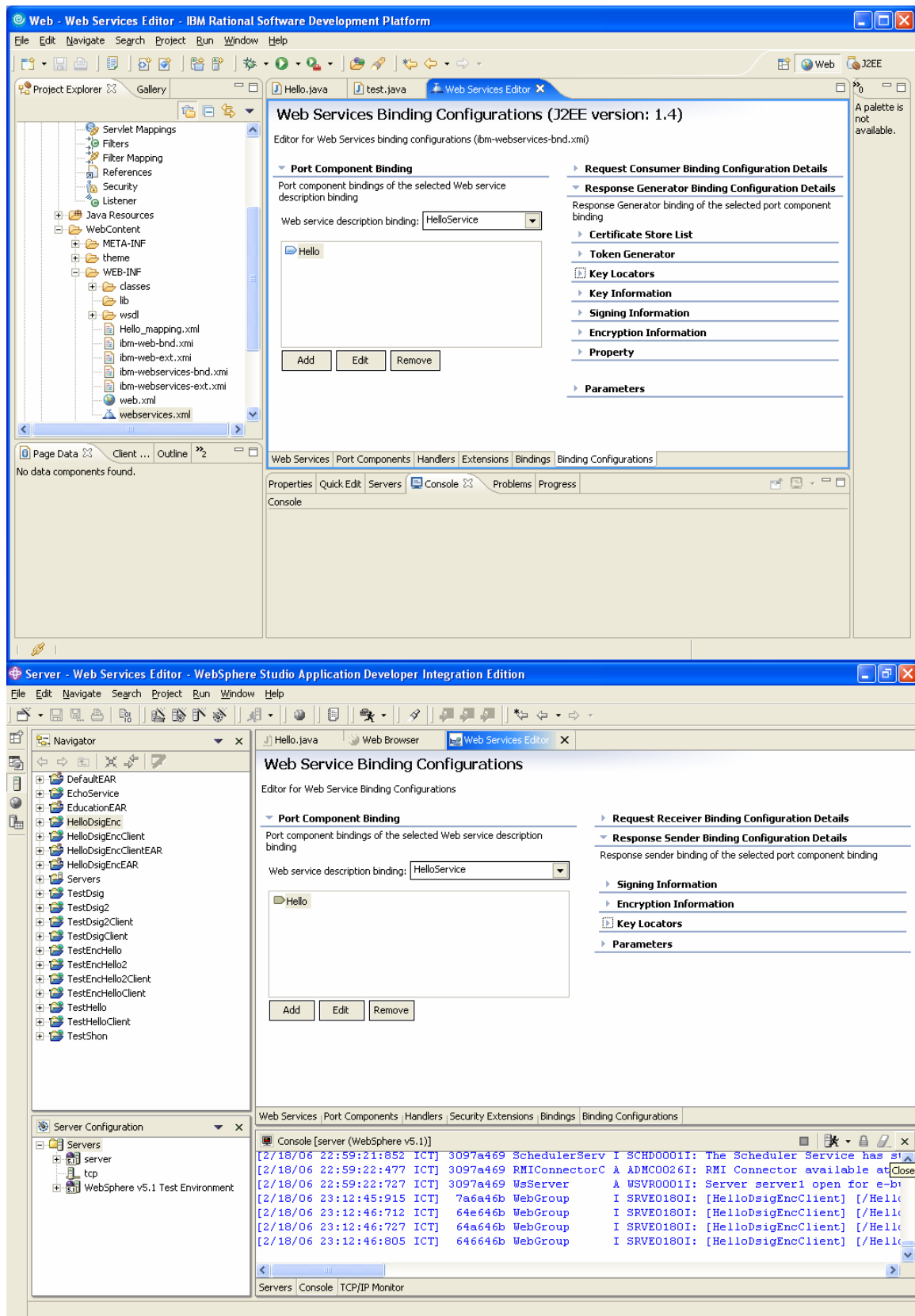
รูปที่ 4.52 แสดงว่ามีการใช้ SSL ใน https protocol

4.3. การทดลองWS-Security กับ เครื่องมืออื่นๆ ดังนี้

1. Websphere Application Server 6.0 กับ Microsoft Visual Studio 2003 with WSE 2.0
2. Websphere Application Developer 5.1 กับ Microsoft Visual Studio 2003 with WSE 2.0
3. Rational Application Developer 6.0 กับ Microsoft Visual Studio 2003 with WSE 2.0
4. Eclipse 3.1 with Apache Tomcat กับ Microsoft Visual Studio 2003

จากผลการทดลองสามารถสรุปรูปแบบของ WS-Security ของแต่ละระบบได้ดังนี้

1. หากไม่มีการใช้ WS-Security สามารถส่งข้อมูลถึงเว็บเซอร์วิสได้หมด
2. หากนำ WS-Security มาใช้ จำเป็นต้องทำให้การ Encryption ตรงตามมาตรฐานของ WS-Security 2004 ซึ่งระบุไว้ใน OASIS ซึ่งมีความละเอียดอ่อนในการระบุชนิดของข้อมูลใน element ต่างๆ โดยต้องระบุตรงกันทั้ง Client และ Server
3. สิ่งสำคัญที่จำเป็นต้องใช้คือ Key และ Certificate ที่สร้างขึ้นมาเพื่อใช้ในการจัดการเรื่องความปลอดภัยบนเว็บเซอร์วิสต้องทำให้ สามารถนำไปใช้ได้ ทั้งใน Java และ Microsoft .Net
4. ถึงแม้จะมีหลุดออกมาให้ทดลองมากมายแต่ทุกที่ นำไปใช้ได้จริงๆ และสามารถข้ามแพลตฟอร์มระหว่าง .Net และ Java ได้ นั่นมีน้อยมาก ซึ่งในขณะนี้ tool ชนิดที่พร้อม และรองรับ WS-Security 2004 ได้แค่ Rational Websphere Application Developer 6.0 และ Microsoft Visual Studio 2003 with WSE 2.0 และ Microsoft Visual Studio 2005 with WSE 3.0 ซึ่งทุกเหล่านี้นี้มีความต้องการทรัพยากรเครื่องในระดับที่สูงพอสมควร อีกทั้งรูปแบบการ configure ที่แตกต่างกันทำให้เกิดการสับสนอย่างมาก
5. Microsoft .Net ค่อนข้างเสถียร ซึ่งเมื่อมีการเปลี่ยนแปลงหรือแก้ไข Security แทนจะไม่ต้องแก้ไข code หรือโปรแกรม เลย ต่างจาก Java หากมีการเปลี่ยนแปลงอาจต้องสร้างโปรเจกใหม่จึงจะรันได้



รูปที่ 4.53 จะเห็นว่า Rational & Websphere มีส่วนคล้ายกันมาก แต่สนับสนุน Java ต่างเวอร์ชันกัน

Token Generator dialog

Token generator name: gen_signtgen

Token generator class: com.ibm.wsspi.wssecurity.token.X509TokenGenerator

Security token:

☒ Use value type

Value type: gen_signtgen_vtype

Local name: http://docs.oasis-open.org/wss/2004/01/oasis-200401-wss-x509-token-profile-1.0#X509

URI:

Call back handler: com.ibm.wsspi.wssecurity.auth.callback.X509CallbackHandler

User ID:

Password:

☒ Use key store

Key store storepass: server

Key store path: \${USER_INSTALL_ROOT}/etc/ws-security/samples/dsig-receiver.ks

Key store type: JKS

Key:

Alias	Key pass	Key name
soapprovider	server	CN=SOAPProvider, OU=TRL, O...

Add Remove

Call back handler property:

Name	Value

Add Remove

Property:

Name	Value

Add Remove

OK Cancel

รูปที่ 4.54 ส่วนที่เพิ่มเข้ามาใน Rational 6.0 คือ Token Generator ซึ่งไม่มีในชุดอื่นๆ

4.5. เปรียบเทียบการทำงานของเว็บเซอร์วิสจาวาและไมโครซอฟท์ดอทเน็ต

ได้มีการทดสอบประสิทธิภาพการทำงานของ Web Service J2EE และ Web Service Microsoft.Net โดยบริษัท Sun Microsystems ซึ่งผลที่ได้ปรากฏว่า Microsoft .NET สามารถทำคะแนนได้ดีกว่า โดย จาวาเองก็โดดเด่นในบางเรื่องด้วยเช่นกัน จาวามีชุดเอพีไอที่ใช้จัดการ XML ได้ โดยตรง สนับสนุนการใช้งานร่วมกับ SOAP, UDDI และ WSDL แต่งานส่วนใหญ่ยังต้องเขียนขึ้นเอง ซึ่งเป็นงานที่ต้องใช้เวลาพอสมควร ในขณะที่การพัฒนาเว็บเซอร์วิสบน Microsoft.Net นั้นมีเครื่องมือหลายอย่างที่จะช่วยในการทำงาน ลดเวลาที่ต้องใช้ในการทำงานและทำได้ง่ายมากกว่า J2EE พอสมควร

ตามที่กล่าวข้างต้นแล้ว J2EE มีจุดเด่นที่สามารถทำงานได้บนฮาร์ดแวร์และระบบปฏิบัติการทุกชนิด ไม่ว่าจะเป็นวินโดวส์ ยูนิกซ์ ลินุกซ์ พีซี จนถึง เมนเฟรม เพียงแต่ต้องมี Java Runtime

Environment บนแพลตฟอร์มเท่านั้น ดังนั้นเรื่องพอร์ตเทบิลิตี้ของ J2EE ก็เป็นเรื่องที่เป็นไปได้จริง แต่ในทางปฏิบัติก็ยังมีจุดอ่อนที่อาจทำให้เกิดปัญหาได้ เนื่องจาก J2EE เป็นเพียงมาตรฐานที่ Sun ออกแบบขึ้นมาเท่านั้น หากผู้พัฒนาซอฟต์แวร์ไม่ว่าจะเป็น IBM, Oracle, BEA หรือ Sun เอง ไม่ได้ปฏิบัติตามมาตรฐานอย่างเคร่งครัด พอร์ตเทบิลิตี้ที่เราเคยคิดว่ามีก็อาจไม่เป็นอย่างที่คาดก็ได้

ถ้าพิจารณา Microsoft.Net เนื่องจาก .Net ทำงานบนวินโดวส์ ดังนั้นจึงไม่มีปัญหาเรื่องการสนับสนุนฮาร์ดแวร์ แต่การทำงานบนแพลตฟอร์มอื่น ปัจจุบันยังเป็นไปไม่ได้ แม้ว่าไมโครซอฟท์บอกว่าจะพยายามพัฒนา COM เพื่อนำไปใช้บนระบบปฏิบัติการอื่นก็ตาม

ดังนั้นปัจจัยสำคัญที่ต้องพิจารณาเรื่องพอร์ตเทบิลิตี้ก็คือ แพลตฟอร์ม หากงานที่ทำมีโอกาสเกี่ยวข้องกับลูกค้าที่ใช้งานแพลตฟอร์มหลาย J2EE น่าจะเหมาะสมกว่าแต่ถ้าไม่ทั้ง J2EE และ .Net ก็ตัวเลือกที่ใกล้เคียงกัน ต้องพิจารณาปัจจัยอื่นด้วย